ATB(Adaptor Template for SpringBoot) User Guide

Table of Contents

1. ATB란? . . . 2
2. 연계 패턴 . . . 2
3. ATB의 특징 . . . 3
4. ATB Adaptor 유형 . . . 3
    4.1 Pub/Sub 방식 . . . 3
    4.2 Provider . . . 4
    4.3 Batch . . . 4
5. ATB DEMO . . . 5
    5.1 DEMO의 설치 & 구동 . . . 5
    5.2 프로젝트의 목록 . . . 5
6 일반 디비to디비 . . . 6
    6.1 개요 . . . 6
    6.2 일반 디비to디비 처리 구성도 . . . 6
    6.3 ATB_DTD_S_01 : 일반 디비to디비 (송신) . . . 6
    6.4 ATB_DTD_R_01 : 일반 디비to디비 (수신) . . . 12
    6.5 재처리 기능 . . . 17
7 대용량 디비to디비 . . . 17
    7.1 개요 . . . 17
    7.2 처리 구성도 . . . 18
    7.3 ATB_DFTFD_S_01 : 대용량 DB TO DB(송신) . . . 19
    7.4 ATB_DFTFD_R_01: 대용량 DB TO DB(수신) . . . 25
    7.5 재처리 기능 . . . 30
8 파일to파일 . . . 31
    8.1 개요 . . . 31
    8.2 파일to파일 처리 구성도 . . . 31
    8.3 ATB_FTF_S_01 : 파일to파일 (송신) . . . 32
    8.4 ATB_FTF_R_01 : 파일to파일(수신) . . . 37
    8.5 재처리 기능 . . . 39
9 파일to디비 . . . 40
    9.1 개요 . . . 40
    9.2 Batch 방식 처리 구성도 . . . 40
    9.3 ATB_FTD_B_01 (배치) . . . 41
    9.4. 재처리 기능 . . . 45
    9.5 PUB/SUB 방식 처리 구성도 . . . 46
    9.6 ATB_FTFD_S_01 : PUB/SUB(송신) . . . 47
    9.7 ATB_FTFD_R_01 : PUB/SUB(수신) . . . 51
10 REST API 프로바이더 (Direct) . . . 59
    10.1 개요 . . . 59
    10.2 DIRECT DB 처리 구성도 . . . 59
    10.3 ATB_REST_PRV_DIRECT_DB : DIRECT DB . . . 59

10.4 DIRECT TCP/IP 처리 구성도 64
10.5 ATB_REST_PRV_DIRECT_NA : DIRECT TCP/IP 64
11 REST API 프로바이더 (Remote) 68
11.1 개요 68
11.2 REMOTE DB 처리 구성도 (좌측 Provider Adaptor) 69
11.3 ATB_REST_PRV_REMOTE_DB : Provider 69
11.4 REMOTE TCP/IP 처리 구성도 (좌측 Provider Adaptor) 71
11.5 ATB_REST_PRV_REMOTE_NA : Provider 72
12 REMOTE 어댑터 75
12.1 개요 75
12.2 REMOTE DB 처리 구성도(우측 Remote Adaptor) 75
12.3 ATB_REMOTE_DB_01 : REMOTE DB 76
12.4 REMOTE TCP/IP 처리 구성도(우측 Remote Adaptor) 81
12.5 NA_REMOTE_TCP_01 : REMOTE TCP/IP 81
13 시뮬레이터 85
13.1 개요 86
13.2 TCP/IP 처리 구성도 86
13.3 NA_TCP_SIMUL_01 : TCP/IP SIMULATOR 86
14 인터페이스 등록 및 Adaptor 연동 89
14.1 auto-config 활성화 89
14.2 인터페이스 등록 89

Author : clupine <clupine@metabuild.co.kr>

1. ATB란?

ATB란 Adaptor Template for SpringBoot의 약자로 SpringBoot를 이용하여 개발을 할 때 자주 개발되는 연계 패턴을 정리하여 템플릿으로 제공하는 것을 목적으로 합니다.

2. 연계 패턴

ATB는 다음과 같은 연계패턴으로 구성되어 있습니다.

순번	구성요소	설명
1	일반 DB TO DB	송신 DB에서 수신 DB로 데이터를 전송하는 연계 *-마스터/디테일 테이블 연계 기능 추가 v1.0.2부터 지원*
2	대용량 DB TO DB	송신 DB에서 수신 DB로 처리과정을 대용량을 처리할 수 있도록 파일로 변환하여 전송하는 연계 *-마스터/디테일 테이블 연계 기능 추가 v1.0.2부터 지원*
3	FILE TO FILE	송신 서버의 저장소 디렉토리에서 수신 서버의 저장소 디렉토리로 파일을 전송하는 연계5
4	FILE TO DB	송신 서버의 저장소 디렉토리에서 수신 서버의 저장소 디렉토리로 파일을 전송하고 수신 서버의 DB로 데이터를 전송하는 연계

5	FILE TO DB 배치	송수신 서버를 분리하지 않고 디렉토리에서 파일을 읽어서 디비로 전송하는 배치 방식의 연계
6	REST PROVIDER(DB)	REST Server에서 수신한 데이터를 DB로 CRUD(Create, Read , Update , Delete)처리 하는 연계 (Remote , Direct DB) 방식으로 구성
7	REST PROVIDER(Remote DB)	위의 구성에서 Provider와 DB 처리 영역을 어뎁터별로 분리되어 연계를 합니다.
8	REST PROVIDER(TCP/IP)	REST Server에서 수신한 데이터를 TCP/IP로 전송하는 연계 , TCP/IP통신 모듈 (NetworkAdaptor)와의 연동 (Remote , Direct) 방식으로 구성
9	REST PROVIDER(Remote TCP/IP)	위의 구성에서 Provider와 TCP/IP 처리 영역을 어뎁터별로 분리되어 연계를 합니다.

패턴은 향후 다양해집니다.

3. ATB의 특징

ATB는 다음과 같은 특징이 있습니다.

- Spring Boot을 기반으로 개발되어 있어서 Spring Boot의 모든 기능을 사용할 수 있습니다.
- 설정 간소화
- 재처리 기능 포함
- 파일 전송모듈 : IFTP를 사용하여 파일을 전송
- TCP/IP : TCP/IP 통신모듈인 NetworkAdaptor 내장
- 인터페이스별 스케줄링 기능
- 연계 이력: 개량된 Single Route SA를 사용하여 로그에 기록
- 이중화 기능 구현(Active/Standby)
- *인터페이스 관리 페이지(인터페이스 등록/설정,담당자 관리 ,Console) 기능 v1.0.2부터 지원*
- *유지보수를 위한 Console 관리 기능 v1.0.2부터 지원*

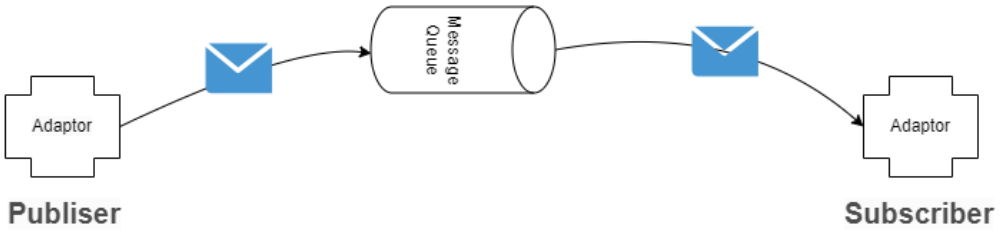
4. ATB Adaptor 유형

이번장에서는 ATB의 어댑터에 유형에 대해서 설명합니다.

4.1 Pub/Sub 방식

어댑터는 Publisher(게시자)와 Subscriber(구독자)로 구분되어 연계를 수행합니다.

Publiser(게시자)와 Subscriber(구독자)

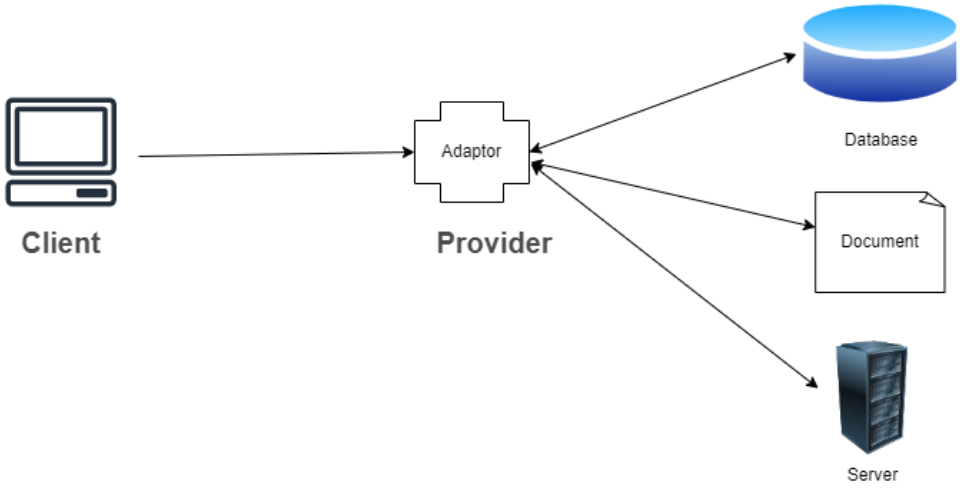


	연계 패턴 DB TO DB , DB TO FILE , FILE TO FILE , FILE TO DB은 Pub/Sub 구조로 되어 있습니다.

4.2 Provider

어댑터는 제공자가 되어서 데이터를 제공하는 역할을 합니다.

Provider(제공자)

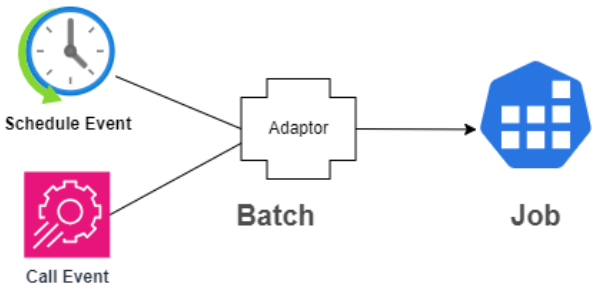


	연계 패턴 Rest API Provder(Direct) , (Remote) 가 Provider 구조로 되어 있습니다.

4.3 Batch

어댑터는 배치로 구동되어서 연계를 수행합니다.

Batch(배치)



연계 패턴 DB TO DB 송신, DB TO FILE 송신 및 Batch , FILE TO FILE 송신 , FILE TO DB 송신 구현부에서 Batch 구조로 되어 있습니다.

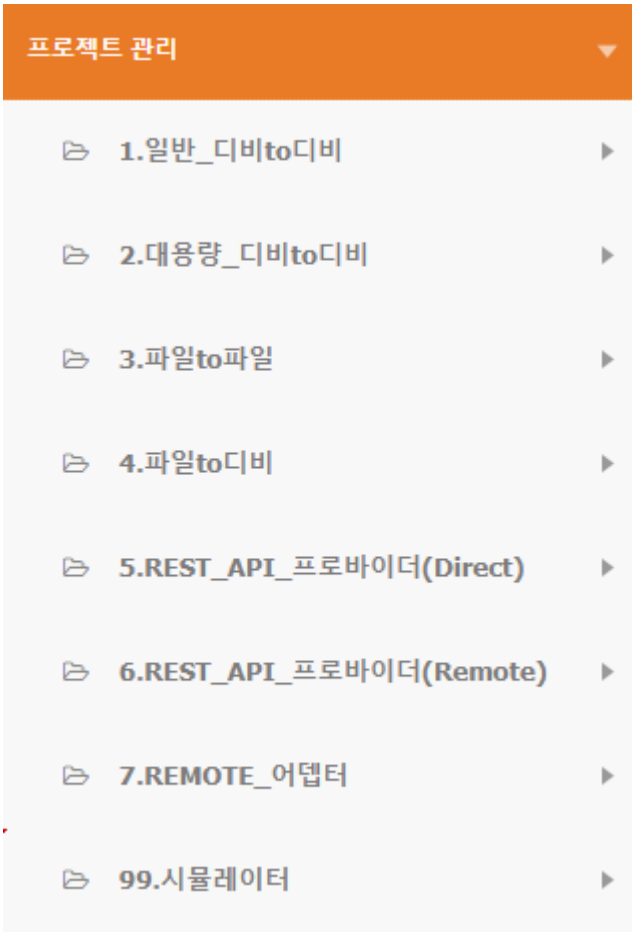
5. ATB DEMO

ATB DEMO는 본 가이드의 시연및 교육를 목적으로 미리 설정된 어댑터 환경을 제공하고 직접 사용하고 체험하여 ATB의 연계 패턴의 기능을 익힐 수 있도록 구성되어 있습니다. 본 개발가이드는 DEMO를 기준으로 설명합니다.

5.1 DEMO의 설치 & 구동

1. 제공받은 DEMO 파일을 압축을 해제하여 ESB DEMO를 구동합니다.
2. http://127.0.0.1:24050/indigoesb 로 접속하여 로그인 합니다.
3. admin / 1234qwer!@#$ 으로 로그인 합니다.

5.2 프로젝트의 목록



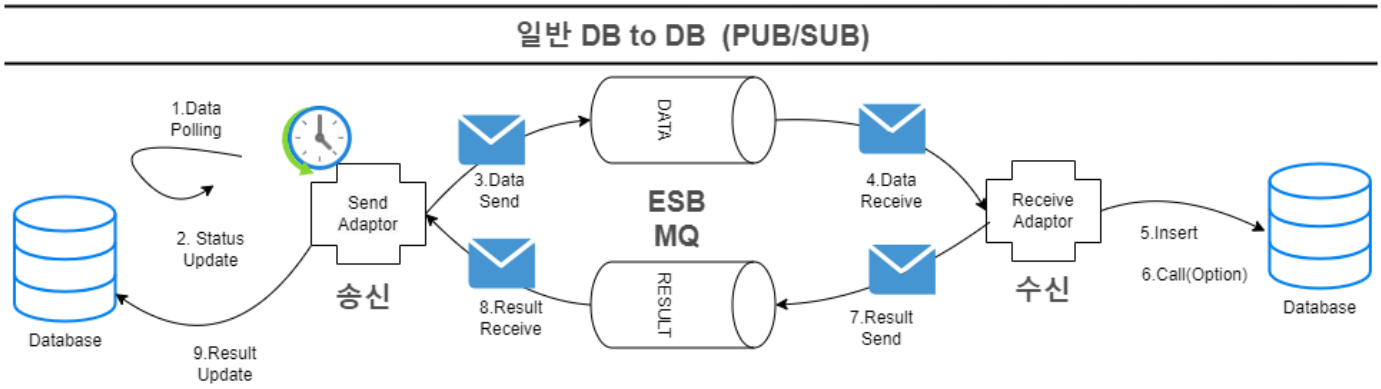
프로젝트를 어댑터를 패턴별과 용도별로 분리를 하였습니다

6 일반 디비to디비

6.1 개요

일반 DB TO DB 어댑터는 송신과 수신 어댑터로 구성되어있으며 구체적인 처리 동작과 설정을 다룹니다.

6.2 일반 디비to디비 처리 구성도



송신 어댑터는 스케줄 이벤트에따라
1. 출발지(Source) DB 테이블을 Polling(조회)한후 조회한 데이터를
2. 연계상태 컬럼을 진행중(P)으로 업데이트하고
3. ESB MQ(MessageQueue) DATA 큐로 전송합니다.
★ 수신 어댑터는 ESB MQ(MessageQueue)인 DATA 큐로 부터
4. Polling(조회)된 데이터를 수신하고
5. 대상(Target) DB 테이블에 INSERT를 한후
6. 옵션에 따라 프로시저를 호출 합니다.
7. 수행 처리 결과를 ESB MQ(MessageQueue)인 RESULT 큐로 전송합니다.
★ 송신 어댑터는
8. 처리결과를 ESB MQ(MessageQueue)로부터 수신하고
9. 처리결과를 출발지(Source) DB 테이블에 UPDATE합니다.

6.3 ATB_DTD_S_01 : 일반 디비to디비 (송신)

- 필수 공통 리소스

DataSource

데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

JmsConnectionFactory

JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

application.yml

어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: DTD_SENDER_01
    description: "(DB SENDER) + JMS + (DB RECEIVER)"
    prefix: DTD
    auto-config: false
    context:
      - interface-id: IF_DTD_001
        action-type: SENDER
        strategy: StandardDTDStrategy
        cron-expression:
            - 0 4 * * * ?
            - 0 8 * * * ?
        on-signal-loop: false
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: DTD_RECEIVER_01
      - interface-id: IF_DTD_003
        action-type: SENDER
        strategy: StandardDTDStrategy
        cron-expression: 0/20 * * * * ?
        on-signal-loop: false
        detail-data: true
        detail-names: CHILD
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: DTD_RECEIVER_01
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 1. atb.if-config

Key	Value	Detail
id	ex) DTD_SENDER_01	ATB 식별 ID
description	ex) "(DB SENDER) + JMS + (DB RECEIVER)"	연계 설명
prefix	ex) DTD	식별자

Table 2. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_DTD_001	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardDTDStrategy	전략 컴포넌트 ID
cronExpression	ex) 0/20 * * * * ?	스케줄 CRON 문법
on-signal-loop	true, false	데이터가 존재할 경우 반복 여부 (default : false)
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) DTD_RECEIVER_01	수신자 ATB 식별 ID
detail-data	ex) true	마스터/디테일 테이블 사용여부 *v1.0.2부터 지원*
detail-names	ex) CHILD,CHILD2	디테일 테이블 Alias(List) *v1.0.2부터 지원*

Table 3. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 4. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true" />
        <setting name="useGeneratedKeys" value="true" />
        <setting name="defaultExecutorType" value="BATCH" />
        <setting name="defaultStatementTimeout" value="10000" />
    </settings>
    <mappers>
        <mapper resource="SQL_IF_DTD_001.xml" />
        <mapper resource="SQL_IF_DTD_003.xml" />
    </mappers>
</configuration>
```

SQL문을 설정으로 관리하기위하여 Mybatis를 사용하는데 필요한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_DTD_001.xml

Mybatis 설정 파일 (단일 테이블 구조)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_DTD_001">
    <select id="SELECT" parameterType="hashmap" resultType="hashmap">
        SELECT *
        FROM IF_DTD_001_S
        WHERE ESB_STATUS_CD = 'N'
          AND ROWNUM <![CDATA[ <= ]]> 2000
    </select>

    <select id="SELECT_RESEND" parameterType="hashmap" resultType="
hashmap">
        SELECT *
        FROM IF_DTD_001_S
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </select>

    <update id="UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_001_S
        SET ESB_STATUS_CD      = 'P',
            ESB_TRANSACTION_ID = #{transactionId},
            ESB_INTERFACE_ID   = #{interfaceId}
        WHERE ESB_SEQ = #{ESB_SEQ}
    </update>

    <update id="RESULT_UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_001_S
        SET ESB_STATUS_CD = #{receiverResultCd},
            ESB_RECV_DT   = #{receiverMsgRecvDt}
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </update>
</mapper>
```

SQL_IF_DTD_003.xml

Mybatis 설정 파일 (마스터/디테일 구조) *v1.0.2부터 지원*

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_DTD_003">
    <select id="SELECT" parameterType="hashmap" resultType="hashmap">
        SELECT *
        FROM IF_DTD_003_S
        WHERE ESB_STATUS_CD = 'N'
          AND ROWNUM <![CDATA[ <= ]]> 2000
    </select>

    <select id="SELECT_CHILD" parameterType="hashmap" resultType="hashmap
">
        SELECT *
        FROM IF_DTD_003_S_CHILD
        WHERE CAR_MODEL = #{CAR_MODEL}
    </select>

    <select id="SELECT_RESEND" parameterType="hashmap" resultType="
hashmap">
        SELECT *
        FROM IF_DTD_003_S
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </select>

    <update id="UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_003_S
        SET ESB_STATUS_CD      = 'P',
            ESB_TRANSACTION_ID = #{transactionId},
            ESB_INTERFACE_ID   = #{interfaceId}
        WHERE ESB_SEQ = #{ESB_SEQ}
    </update>

    <update id="RESULT_UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_003_S
        SET ESB_STATUS_CD = #{receiverResultCd},
            ESB_RECV_DT   = #{receiverMsgRecvDt}
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 SELECT , SELECT_RESEND, UPDATE, RESULT_UPDATE 를 추가합니다.

6.4 ATB_DTD_R_01 : 일반 디비to디비 (수신)

- 필수 공통 리소스

  DataSource

  데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: DTD_RECEIVER_01
    description: "(DB SENDER) + JMS + (DB RECEIVER)"
    prefix: DTD
    auto-config: false
    context:
      - interface-id: IF_DTD_001
        action-type: RECEIVER
        strategy: StandardDTDStrategy
        procedure-call: true
      - interface-id: IF_DTD_003
        action-type: RECEIVER
        strategy: StandardDTDStrategy
        procedure-call: true
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 5. atb.if-config

Key	Value	Detail
id	ex) DTD_RECEIVER_01	ATB 식별 ID
description	ex) "(DB SENDER) + JMS + (DB RECEIVER)"	연계 설명
prefix	ex) DTD	식별자

Key	Value	Detail
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 6. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_DTD_001	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardDTDStrategy	전략 컴포넌트 ID

Table 7. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_DTD_001	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardDTDStrategy	전략 컴포넌트 ID
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
procedure-call	ex) true	프로시저 호출 여부 (default : false)

Table 8. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 9. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true" />
        <setting name="useGeneratedKeys" value="true" />
        <setting name="defaultExecutorType" value="BATCH" />
        <setting name="defaultStatementTimeout" value="10000" />
    </settings>
    <mappers>
        <mapper resource="SQL_IF_DTD_001.xml" />
        <mapper resource="SQL_IF_DTD_003.xml" />
    </mappers>
</configuration>
```

SQL문을 설정으로 관리하기위하여 Mybatis를 사용하는데 필요한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_DTD_001.xml

Mybatis 설정 파일 (단일 테이블 구조)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_DTD_001">
    <insert id="INSERT" parameterType="hashmap">
        INSERT INTO IF_DTD_001_T( ESB_SEQ
                                , ESB_STATUS_CD
                                , ESB_TRANSACTION_ID
                                , ESB_RECV_DT
                                , ESB_INTERFACE_ID
                                , FIRST_NAME
                                , LAST_NAME
                                , EMAIL
                                , PHONE
                                , CUSTOMER_ID)
        values (SEQ_IF_DTD_001_T.NEXTVAL,
                'N',
                #{transactionId     ,jdbcType=VARCHAR},
                #{receiverMsgRecvDt     ,jdbcType=VARCHAR},
                #{interfaceId     ,jdbcType=VARCHAR},
                #{FIRST_NAME     ,jdbcType=VARCHAR},
                #{LAST_NAME     ,jdbcType=VARCHAR},
                #{EMAIL     ,jdbcType=VARCHAR},
                #{PHONE     ,jdbcType=VARCHAR},
                #{CUSTOMER_ID     ,jdbcType=INTEGER})
    </insert>

    <update id="CALL" statementType="CALLABLE">{CALL
SP_IF_DTD_001_T(#{transactionId ,jdbcType=VARCHAR})}</update>
</mapper>
```

SQL_IF_DTD_003.xml

Mybatis 설정 파일 (마스터/디테일 구조) *v1.0.2부터 지원*

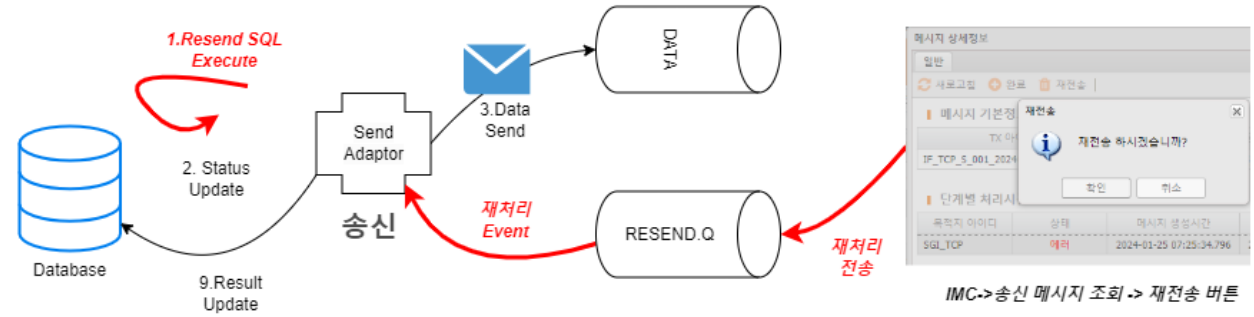
```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_DTD_003">
    <insert id="INSERT" parameterType="hashmap">
        INSERT INTO IF_DTD_003_T(        ESB_SEQ
                                        ,ESB_STATUS_CD
                                        ,ESB_TRANSACTION_ID
                                        ,ESB_RECV_DT
                                        ,ESB_INTERFACE_ID
                                        ,CAR_MODEL
                                        ,CAR_YEAR
                                        ,CAR_BRAND
                                        ,CAR_VIN )
        values (
                 SEQ_IF_DTD_003_T.NEXTVAL ,
                 'N',
                 #{transactionId     ,jdbcType=VARCHAR},
                 #{receiverMsgRecvDt     ,jdbcType=VARCHAR},
                 #{interfaceId     ,jdbcType=VARCHAR},
                 #{CAR_MODEL     ,jdbcType=VARCHAR},
                 #{CAR_YEAR     ,jdbcType=VARCHAR},
                 #{CAR_BRAND     ,jdbcType=VARCHAR},
                 #{CAR_VIN     ,jdbcType=VARCHAR}
                 )
    </insert>

    <insert id="INSERT_CHILD" parameterType="hashmap">
        INSERT INTO IF_DTD_003_T_CHILD( CAR_SEQ
                                      , CAR_MODEL
                                      , AUDIO
                                      , COLOR
                                      , TIRES)
        values (#{CAR_SEQ     ,jdbcType=INTEGER},
                #{CAR_MODEL     ,jdbcType=VARCHAR},
                #{AUDIO     ,jdbcType=VARCHAR},
                #{COLOR     ,jdbcType=VARCHAR},
                #{TIRES     ,jdbcType=VARCHAR})
    </insert>

    <update id="CALL" statementType="CALLABLE">{CALL
SP_IF_DTD_003_T(#{transactionId ,jdbcType=VARCHAR})}</update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 INSERT와 CALL문을 추가합니다. 송신(Sender) 에서 발송한 데이터의 키를 Value에 입력합니다. DB to DB같은경우는 송신 DB의 컬럼이 될것입니다. transactionId(트랜젝션 ID),receiverMsgRecvDt(수신처리시간),interfaceId(인터페이스ID)는 ATB 에서 고정된 예약된 값으로 사용됩니다.

6.5 재처리 기능

- IMC / 메시지 이력관리 / 송수신 메시지 조회 / 메시지 상세정보 / 실패한 목적지 아이디를 선택후 / 재전송 버튼을 클릭하면 재전송이 가능합니다.

일반 DB to DB (송신 재전송)



IMC->송신 메시지 조회 -> 재전송 버튼

```
<select id="SELECT_RESEND" parameterType="hashmap" resultType="hashmap">
    SELECT * FROM IF_DTD_001_S
    WHERE ESB_TRANSACTION_ID = #{transactionId}
</select>
```

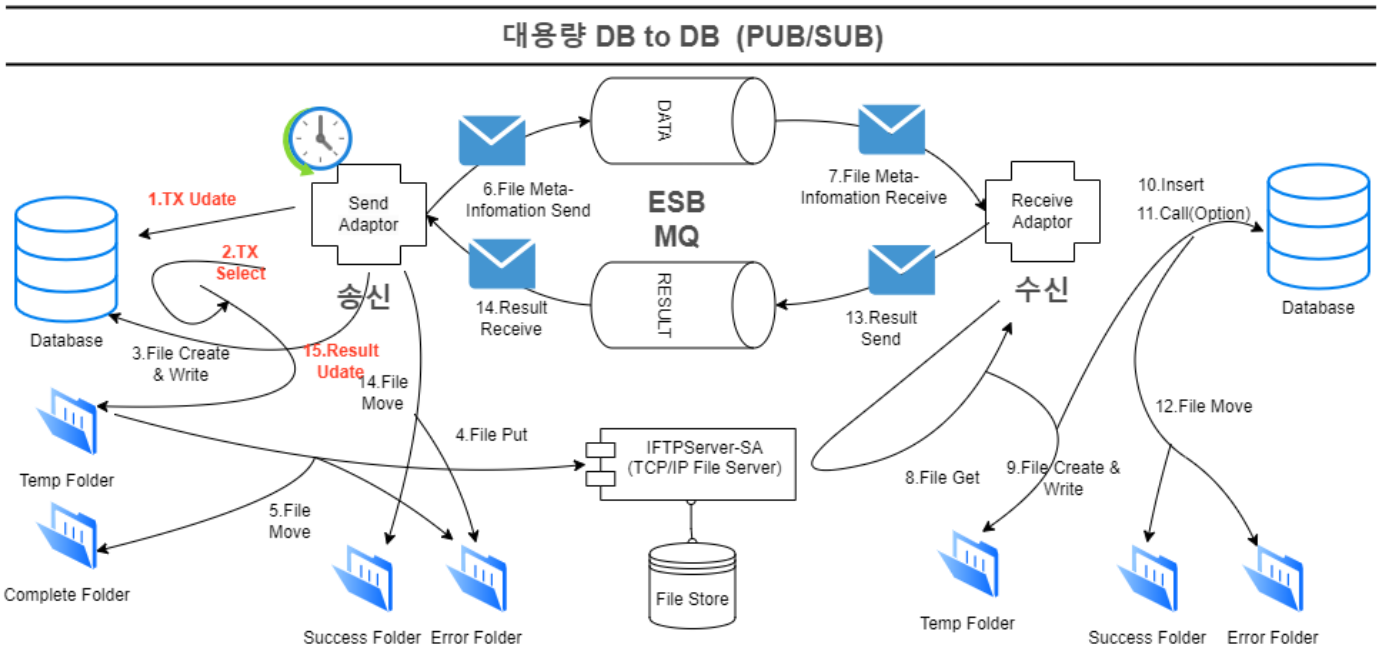
재처리 기능은 송신 어댑터에서 구현되어 작동되며 재처리 SQL을 실행합니다. 이후 발생되는 프로세스는 동일하게 동작합니다.

7 대용량 디비to디비

7.1 개요

대용량 DB TO DB

7.2 처리 구성도



★ 송신 어댑터는 스케줄 이벤트에따라
1. 출발지(Source) DB 테이블을 연계 상태 컬럼을 N인 데이터를 상태코드를 'P'(처리중)과 트랜젝션ID을 업데이트 2. 트랜젝션ID로 조회한 데이터를
3. 임시(Temp) 폴더에 파일을 생성하고 기록합니다.
4. 생성된 파일을 ESB Hub의 파일서버로 전송(PUT) 하고
5. 전송 실패하였다면 에러(Error) 폴더에 파일을 넣고 종료를 합니다. 정상적으로 송신하였다면.
6. 완료(Complete) 폴더 에 넣고 ESB MQ(MessageQueue) DATA 큐로 파일의 메타정보를 전송합니다.
★ 수신 어댑터는 ESB MQ(MessageQueue)인 DATA 큐로 부터
7. 파일의 메타정보를 수신하고 ESB Hub의 파일서버로 부터 임시(Temp) 폴더에 파일 수신(GET) 을 진행합니다.
8. 수신 받은 파일을 대상(Target) DB 테이블에 INSERT를 한후
9. 옵션에 따라 프로시저를 호출 합니다.
10. 처리 결과에 따라 파일을 성공(Success) 또는 에러(Error) 폴더에 이동시키고 처리 결과를 ESB MQ(MessageQueue)인 RESULT 큐로 전송합니다.
★ 송신 어댑터는
11. ESB MQ(MessageQueue)로부터 수신하고 처리결과를 출발지(Source) DB 테이블에 UPDATE합니다.
12. 처리결과를 완료(Complete) 폴더에 있던 파일을 성공(Success) 또는 에러(Error) 폴더에 이동시킵니다.

기존의 대용량 DB to DB는 파일을 처리할때 송신테이블에 대한 결과처리를 하지 못하였으나 연계 메커니즘(1번,2번)을 변경하여 송신테이블에 결과를 업데이트할수 있도록 변경하였습니다.v1.0.2부터 지원

7.3 ATB_DFTFD_S_01 : 대용량 DB TO DB(송신)

- 필수 공통 리소스

  DataSource

  데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: DFTFD_SENDER_01
    description: "(DB + FILE SENDER) + JMS + (FILE + DB RECEIVER)"
    prefix: DFTFD
    auto-config: false
    iftp:
      enabled: true
      data-packet-size: 4096
      retry-data-cnt: 100
      view-count: 1000
      transfer-sleep: 0
      encrypt: false
      remote-hosts:
        - clupine.duckdns.org:24052
    context:
      - interface-id: IF_DTD_002
        action-type: SENDER
        strategy: StandardDFTFDStrategy
        cron-expression: 0/20 * * * * ?
        on-signal-loop: false
        file-send-path: "/atb/send/IF_DTD_002"
        file-temp-path: "/atb/send/IF_DTD_002/temp"
        file-complete-path: "/atb/send/IF_DTD_002/complete"
        file-error-path: "/atb/send/IF_DTD_002/error"
        file-success-path: "/atb/send/IF_DTD_002/success"
        file-retry-count: 3
        file-delete: false
        file-type: DELIMITER
        file-delimiter: ','
        file-delimiter-qualifier: '"'
        send-system-code: A01
```

```yaml
        receive-system-code: B01
        receiver-ids: DFTFD_RECEIVER_01
      - interface-id: IF_DTD_004
        action-type: SENDER
        strategy: StandardDFTFDStrategy
        cron-expression: 0/20 * * * * ?
        on-signal-loop: false
        file-send-path: "/atb/send/IF_DTD_004"
        file-temp-path: "/atb/send/IF_DTD_004/temp"
        file-complete-path: "/atb/send/IF_DTD_004/complete"
        file-error-path: "/atb/send/IF_DTD_004/error"
        file-success-path: "/atb/send/IF_DTD_004/success"
        file-retry-count: 3
        file-delete: false
        file-type: DELIMITER
        file-delimiter: ','
        file-delimiter-qualifier: '"'
        detail-data: true
        detail-names: CHILD
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: DFTFD_RECEIVER_01
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 10. atb.if-config

Key	Value	Detail
id	ex) DFTFD_SENDER_01	ATB 식별 ID
description	ex) "(DB + FILE SENDER) + JMS + (FILE + DB RECEIVER)"	연계 설명
prefix	ex) DFTFD	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 11. atb.if-config.iftp

Key	Value	Detail
enabled	true, false	iftp 파일서버 클라이언트를 활성화 여부 (default : false)

Key	Value	Detail
data-packet-size	ex) 4096	전송 패킷 사이즈 (default : 4096)
retry-data-cnt	ex) 100	재전송 검증 요청 횟수 (default : 100) 예) 100회 전송후 재전송 검증 요청 전문 발송
view-count	ex) 1000	예) 1000회 전송후 전송 로그 표기 (default : 1000)
transfer-sleep	ex) 0	전송시간 간격 Sleep 여부, 수신 받는 서버가 클라이언트보다 느릴경우 지연처리 (ms)
encrypt	true, false	파일암호화 여부 (default : false)
remote-hosts	ex) clupine.duckdns.org:24052	파일서버 호스트 (IP:PORT) 형식으로 입력

Table 12. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_DTD_002	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardDFTFDStrategy	전략 컴포넌트 ID
cronExpression	ex) 0/20 * * * * ?	스케줄 CRON 문법
actFirst	ture, false	true 일경우 스케줄이 작동하지 않고 기동시 단한번 실행이 됩니다. (default : false)
on-signal-loop	true, false	데이터가 존재할 경우 반복 여부 (default : false)
file-send-path	ex) "/atb/send/IF_DTD_002"	송신 파일 경로
file-temp-path	ex) "/atb/send/IF_DTD_002/temp"	임시 파일 경로
file-complete-path	ex) "/atb/send/IF_DTD_002/complete"	완료 파일 경로
file-error-path	ex) "/atb/send/IF_DTD_002/error"	에러 파일 경로
file-success-path	ex) "/atb/send/IF_DTD_002/success"	성공 파일 경로
file-retry-count	ex) 3	파일 재전송 시도 횟수

Key	Value	Detail
file-delete	true, false	파일 삭제 여부
file-type	ex) DELIMITER	파일의 타입
file-delimiter	ex) ','	Value 와 Value 사이의 구분자
file-delimiter-qualifier	ex) '"'	Value 를 감싸는 구분자
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) DFTFD_RECEIVER_01	수신자 ATB 식별 ID
detail-data	ex) true	마스터/디테일 테이블 사용여부 *v1.0.2부터 지원*
detail-names	ex) CHILD,CHILD2	디테일 테이블 Alias(List) *v1.0.2부터 지원*

Table 13. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 14. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true" />
        <setting name="useGeneratedKeys" value="true" />
        <setting name="defaultExecutorType" value="BATCH" />
        <setting name="defaultStatementTimeout" value="10000" />
         <setting name="callSettersOnNulls" value="true"/>
    </settings>
    <mappers>
        <mapper resource="SQL_IF_DTD_002.xml" />
        <mapper resource="SQL_IF_DTD_004.xml" />
    </mappers>
</configuration>
```

Mybatis를 사용하기위한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_DTD_002.xml

Mybatis 설정 파일 (단일 테이블 구조)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_DTD_004">

    <update id="TX_UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_004_S
        SET ESB_STATUS_CD      = 'P',
            ESB_INTERFACE_ID   = #{interfaceId},
            ESB_TRANSACTION_ID = #{transactionId}
        WHERE ESB_STATUS_CD = 'N'
    </update>

    <select id="SELECT" parameterType="hashmap" resultType=
"java.util.LinkedHashMap">
        SELECT *
        FROM IF_DTD_004_S
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </select>

    <select id="SELECT_CHILD" parameterType="hashmap" resultType="hashmap
">
        SELECT *
        FROM IF_DTD_004_S_CHILD
        WHERE CAR_MODEL = #{CAR_MODEL}
    </select>

    <update id="RESULT_UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_004_S
        SET ESB_STATUS_CD = #{receiverResultCd},
            ESB_RECV_DT   = #{receiverMsgRecvDt}
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </update>

    <select id="SELECT_RESEND" parameterType="hashmap" resultType="
hashmap">
        SELECT *
        FROM IF_DTD_004_S
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </select>
</mapper>
```

SQL_IF_DTD_004.xml

Mybatis 설정 파일 (마스터/디테일 구조) *v1.0.2부터 지원*

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_DTD_002">

    <update id="TX_UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_002_S
        SET ESB_STATUS_CD      = 'P',
            ESB_INTERFACE_ID   = #{interfaceId},
            ESB_TRANSACTION_ID = #{transactionId}
        WHERE ESB_STATUS_CD = 'N'
    </update>

    <select id="SELECT" parameterType="hashmap" resultType=
"java.util.LinkedHashMap">
        SELECT *
        FROM IF_DTD_002_S
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </select>

    <update id="RESULT_UPDATE" parameterType="hashmap">
        UPDATE IF_DTD_002_S
        SET ESB_STATUS_CD = #{receiverResultCd},
            ESB_RECV_DT   = #{receiverMsgRecvDt}
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </update>

    <select id="SELECT_RESEND" parameterType="hashmap" resultType="
hashmap">
        SELECT *
        FROM IF_DTD_002_S
        WHERE ESB_TRANSACTION_ID = #{transactionId}
    </select>

</mapper>
```

namespace는 인터페이스 ID를 넣고 TX_UPDATE , SELECT , RESULT_UPDATE,SELECT_RESEND 를 추가합니다. 기존의 (SELECT , UPDATE, RESULT_UPDATE , SELECT_RESEND) 였던것을 변경함 *v1.0.2부터 지원*

7.4 ATB_DFTFD_R_01: 대용량 DB TO DB(수신)

- 필수 공통 리소스

DataSource

데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

JmsConnectionFactory

JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

application.yml

어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: DFTFD_RECEIVER_01
    description: "(DB + FILE SENDER) + JMS + (FILE + DB RECEIVER)"
    prefix: DFTFD
    auto-config: false
    iftp:
      enabled: true
      data-packet-size: 4096
      retry-data-cnt: 100
      view-count: 1000
      transfer-sleep: 0
      encrypt: false
      remote-hosts:
        - clupine.duckdns.org:24052
    context:
      - interface-id: IF_DTD_002
        action-type: RECEIVER
        strategy: StandardDFTFDStrategy
        procedure-call: true
        file-recv-path: "/atb/recv/IF_DTD_002"
        file-temp-path: "/atb/recv/IF_DTD_002/temp"
        file-success-path: "/atb/recv/IF_DTD_002/success"
        file-error-path: "/atb/recv/IF_DTD_002/error"
        file-retry-count: 3
        file-type: DELIMITER
        file-delimiter: ','
        file-delimiter-qualifier: '"'
        file-first-header: true
        file-error-skip: false
      - interface-id: IF_DTD_004
        action-type: RECEIVER
        strategy: StandardDFTFDStrategy
        procedure-call: true
        file-recv-path: "/atb/recv/IF_DTD_004"
        file-temp-path: "/atb/recv/IF_DTD_004/temp"
```

```
        file-success-path: "/atb/recv/IF_DTD_004/success"
        file-error-path: "/atb/recv/IF_DTD_004/error"
        file-retry-count: 3
        file-type: DELIMITER
        file-delimiter: ','
        file-delimiter-qualifier: '"'
        file-first-header: true
        file-error-skip: false
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 15. atb.if-config

Key	Value	Detail
id	ex) DFTFD_RECEIVER_01	ATB 식별 ID
description	ex) "(DB + FILE SENDER) + JMS + (FILE + DB RECEIVER)"	연계 설명
prefix	ex) DFTFD	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 16. atb.if-config.iftp

Key	Value	Detail
enabled	true, false	iftp 파일서버 클라이언트를 활성화 여부 (default : false)
data-packet-size	ex) 4096	전송 패킷 사이즈 (default : 4096)
retry-data-cnt	ex) 100	데이터 전송후 재시도 횟수 (default : 100)
view-count	ex) 1000	전송시 파일의 뷰 카운트 (default : 1000)
transfer-sleep	ex) 0	전송시간간격 (ms)
encrypt	true, false	파일암호화 여부 (default : false)
remote-hosts	ex) clupine.duckdns.org:24052	파일서버 호스트 (IP:PORT) 형식으로 입력

Table 17. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_DTD_002	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardDFTFDStrategy	전략 컴포넌트 ID
file-recv-path	ex) "/atb/recv/IF_DTD_002"	수신 파일 경로
file-temp-path	ex) "/atb/recv/IF_DTD_002/temp"	임시 파일 경로
file-success-path	ex) "/atb/recv/IF_DTD_002/success"	성공 파일 경로
file-error-path	ex) "/atb/recv/IF_DTD_002/error"	에러 파일 경로
file-retry-count	ex) 3	파일 재전송 시도 횟수 (default : 3)
file-type	ex) DELIMITER	파일의 타입
file-delimiter	ex) ','	Value 와 Value 사이의 구분자
file-delimiter-qualifier	ex) '"'	Value 를 감싸는 구분자
file-first-header	true, false	파일의 헤더가 있는지 여부 (default : true)
fileErrorSkip	true, false	에러 파일을 스킵할지 여부 (default : false)
procedure-call	ex) true	프로시저 호출 여부 (default : false)

Table 18. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 19. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true"/>
        <setting name="useGeneratedKeys" value="true"/>
        <setting name="defaultExecutorType" value="BATCH"/>
        <setting name="defaultStatementTimeout" value="10000"/>
    </settings>
    <mappers>
        <mapper resource="SQL_IF_DTD_002.xml"/>
        <mapper resource="SQL_IF_DTD_004.xml"/>
    </mappers>
</configuration>
```

Mybatis를 사용하기위한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_DTD_002.xml

Mybatis 설정 파일

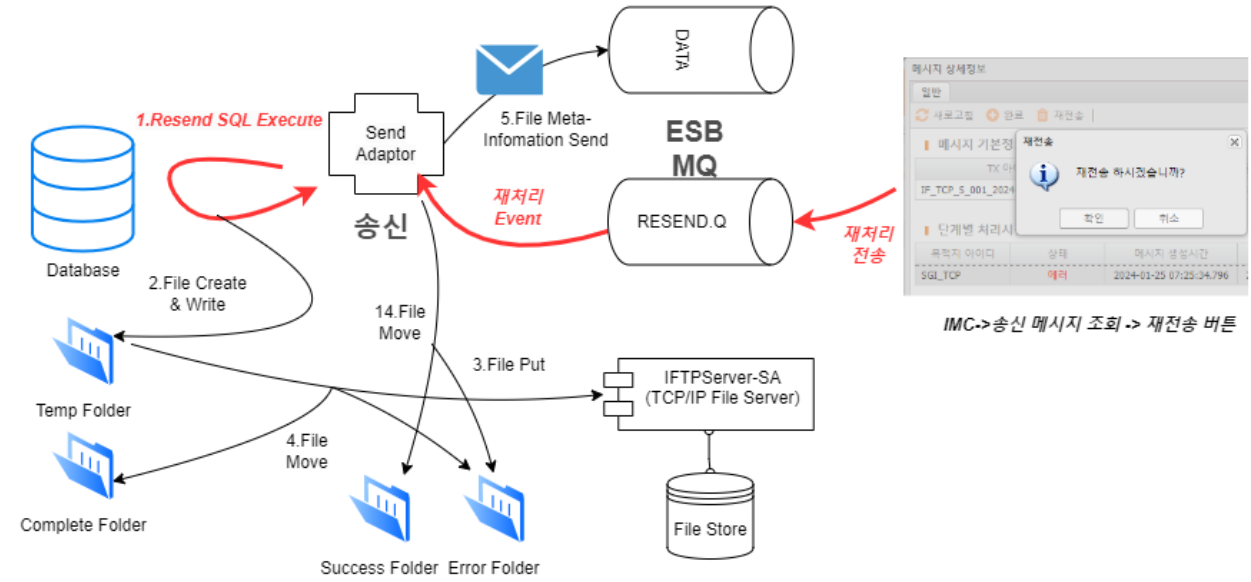
```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_DTD_002">
    <insert id="INSERT" parameterType="hashmap">
        INSERT INTO IF_DTD_002_T( ESB_SEQ
                                , ESB_STATUS_CD
                                , ESB_TRANSACTION_ID
                                , ESB_RECV_DT
                                , ESB_INTERFACE_ID
                                , FIRST_NAME
                                , LAST_NAME
                                , EMAIL
                                , PHONE
                                , CUSTOMER_ID
                                , CREATE_DT)
        values (SEQ_IF_DTD_002_T.NEXTVAL,
                'N',
                #{transactionId     ,jdbcType=VARCHAR},
                #{receiverMsgRecvDt     ,jdbcType=VARCHAR},
                #{interfaceId     ,jdbcType=VARCHAR},
                #{FIRST_NAME     ,jdbcType=VARCHAR},
                #{LAST_NAME     ,jdbcType=VARCHAR},
                #{EMAIL     ,jdbcType=VARCHAR},
                #{PHONE     ,jdbcType=VARCHAR},
                #{CUSTOMER_ID     ,jdbcType=INTEGER},
                #{CREATE_DT     ,jdbcType=VARCHAR})
    </insert>

    <update id="CALL" statementType="CALLABLE">{CALL
SP_IF_DTD_002_T(#{transactionId ,jdbcType=VARCHAR})}</update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 INSERT , CALL 를 추가합니다.

7.5 재처리 기능

- 파라메터가 필요한 재처리는 IMC / 테스트 / JMS테스트 기능을 이용해서 재전송이 가능합니다.

대용량 DB to DB (재전송)




```
<select id="SELECT_RESEND" parameterType="hashmap" resultType="hashmap">
    SELECT * FROM IF_DTD_002_S
    WHERE ESB_TRANSACTION_ID = #{transactionId}
</select>
```

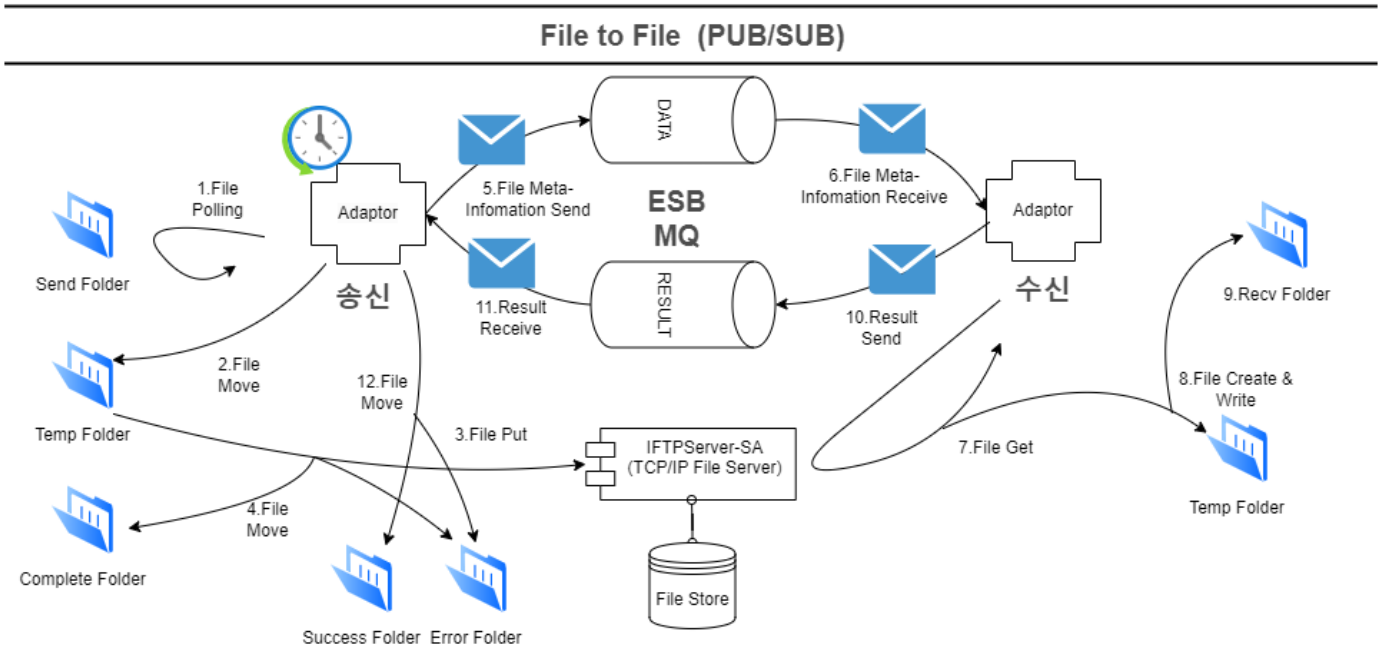
재처리 기능은 송신 어댑터에서 구현되어 작동되며 재처리 SQL을 실행합니다. 이후 발생되는 프로세스는 동일하게 동작합니다.

8 파일to파일

8.1 개요

FILE TO FILE

8.2 파일to파일 처리 구성도



★ 송신 어댑터는 스케줄 이벤트에따라
1. 출발지(Source) 의 송신(Send) 폴더에 파일을 가져온후
2. 송신할 파일을 임시(Temp) 폴더로 이동시킨다.
3. 생성된 파일을 ESB Hub의 파일서버로 전송(PUT) 하고
4. 전송 실패하였다면 에러(Error) 폴더에 파일을 넣고 종료를 합니다.
정상적으로 송신하였다면.
5. 완료(Complete) 폴더 에 넣고 ESB MQ(Message Queue) DATA 큐로 파일의 메타정보를 전송합니다.
★ 수신 어댑터는 ESB MQ(Message Queue)인 DATA 큐로 부터
6. 파일의 메타정보를 수신하고 ESB Hub의 파일서버로 부터 임시(Temp) 폴더에 파일 수신(GET) 을 진행합니다.
7. 정상적으로 수신하였다면 수신(Recv) 폴더 이동하고
8. 처리 결과를 ESB MQ(Message Queue)의 RESULT 큐로 전송합니다.
★ 송신 어댑터는
9. 처리결과를 ESB MQ(Message Queue)로부터 수신하고
10. 처리결과에 따라 완료(Complete) 폴더에 있던 파일을 성공(Success) 또는 에러(Error) 폴더에 이동시킵니다.

8.3 ATB_FTF_S_01 : 파일to파일 (송신)

- 필수 공통 리소스

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

application.yml

어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: "FTF_SENDER_01"
    description: "(FILE SENDER) + JMS and Socket + (FILE RECEIVER)"
    prefix: FTF
    auto-config: false
    iftp:
      enabled: true
      data-packet-size: 8192
      retry-data-cnt: 2000
      view-count: 1000
      transfer-sleep: 0
      encrypt: false
      remote-hosts:
        - clupine.duckdns.org:24052
    context:
      - interface-id: IF_FTF_001
        action-type: SENDER
        strategy: StandardFTFStrategy
        cron-expression: 0/20 * * * * ?
        on-signal-loop: true
        file-send-path: "/atb/send/IF_FTF_001"
        file-temp-path: "/atb/send/IF_FTF_001/temp"
        file-complete-path: "/atb/send/IF_FTF_001/complete"
        file-error-path: "/atb/send/IF_FTF_001/error"
        file-success-path: "/atb/send/IF_FTF_001/success"
        file-send-count: 1
        file-retry-count: 3
        file-delete: false
        file-glob-pattern: "*.{csv,doc,xml,xlsx,zip,hwp,exe,tar}"
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: FTF_RECEIVER_01
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
  autoconfigure:
    exclude:
      -
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
  task:
    scheduling:
      pool:
        size: 10
```

Table 20. atb.if-config

Key	Value	Detail
id	ex) FTF_SENDER_01	ATB 식별 ID
description	ex) "(FILE SENDER) + JMS and Socket + (FILE RECEIVER)"	연계 설명
prefix	ex) FTF	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 21. atb.if-config.iftp

Key	Value	Detail
enabled	true, false	iftp 파일서버 클라이언트를 활성화 여부 (default : false)
data-packet-size	ex) 4096	전송 패킷 사이즈 (default : 4096)
retry-data-cnt	ex) 100	데이터 전송후 재시도 횟수 (default : 100)
view-count	ex) 1000	전송시 파일의 뷰 카운트 (default : 1000)
transfer-sleep	ex) 0	전송시간간격 (ms)
encrypt	true, false	파일암호화 여부 (default : false)
remote-hosts	ex) clupine.duckdns.org:24052	파일서버 호스트 (IP:PORT) 형식으로 입력

Table 22. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_FTF_001	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardFTFStrategy	전략 컴포넌트 ID
cronExpression	ex) 0/20 * * * * ?	스케줄 CRON 문법
on-signal-loop	true, false	데이터가 존재할 경우 반복 여부 (default : false)
file-send-path	ex) "/atb/send/IF_FTF_001"	송신 파일 경로
file-temp-path	ex) "/atb/send/IF_FTF_001/temp"	임시 파일 경로

Key	Value	Detail
file-complete-path	ex) "/atb//send/IF_FTF_001/complete"	완료 파일 경로
file-error-path	ex) "/atb/send/IF_FTF_001/error"	에러 파일 경로
file-success-path	ex) "/atb/send/IF_FTF_001/success"	성공 파일 경로
file-send-count	ex) 1	파일 전송 갯수
file-retry-count	ex) 3	파일 재전송 시도 횟수
file-delete	true, false	파일 삭제 여부
file-glob-pattern	ex) "*.{csv,doc,xml,xlsx,zip,hwp,exe,tar}"	파일 Glob 패턴
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) FTF_RECEIVER_01	수신자 ATB 식별 ID

Table 23. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 24. spring.autoconfigure

Key	Value	Detail
exclude	ex) org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration	제외할 설정

8.4 ATB_FTF_R_01 : 파일to파일(수신)

- 필수 공통 리소스

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: "FTF_RECEIVER_01"
    description: "(FILE SENDER) + JMS and Socket + (FILE RECEIVER)"
    prefix: FTF
    auto-config: false
    iftp:
      enabled: true
      data-packet-size: 8192
      retry-data-cnt: 2000
      view-count: 1000
      transfer-sleep: 0
      encrypt: false
      remote-hosts:
        - clupine.duckdns.org:24052
    context:
      - interface-id: IF_FTF_001
        action-type: RECEIVER
        strategy: StandardFTFStrategy
        file-recv-path: "/atb/recv/IF_FTF_001"
        file-temp-path: "/atb/recv/IF_FTF_001/temp"
        file-retry-count: 3
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
  autoconfigure:
    exclude:
      -
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

Table 25. atb.if-config

Key	Value	Detail
id	ex) FTF_RECEIVER_01	ATB 식별 ID
description	ex) "(FILE SENDER) + JMS and Socket + (FILE RECEIVER)"	연계 설명
prefix	ex) FTF	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 26. atb.if-config.iftp

Key	Value	Detail
enabled	true, false	iftp 파일서버 클라이언트를 활성화 여부 (default : false)
data-packet-size	ex) 4096	전송 패킷 사이즈 (default : 4096)
retry-data-cnt	ex) 100	데이터 전송후 재시도 횟수 (default : 100)
view-count	ex) 1000	전송시 파일의 뷰 카운트 (default : 1000)
transfer-sleep	ex) 0	전송시간간격 (ms)
encrypt	true, false	파일암호화 여부 (default : false)
remote-hosts	ex) clupine.duckdns.org:24052	파일서버 호스트 (IP:PORT) 형식으로 입력

Table 27. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_FTF_001	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardFTFStrategy	전략 컴포넌트 ID
file-recv-path	ex) "/atb/recv/IF_FTF_001"	수신 파일 경로
file-temp-path	ex) "/atb/recv/IF_FTF_001/temp"	임시 파일 경로
file-retry-count	ex) 3	파일 재전송 시도 횟수 (default : 3)

Table 28. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

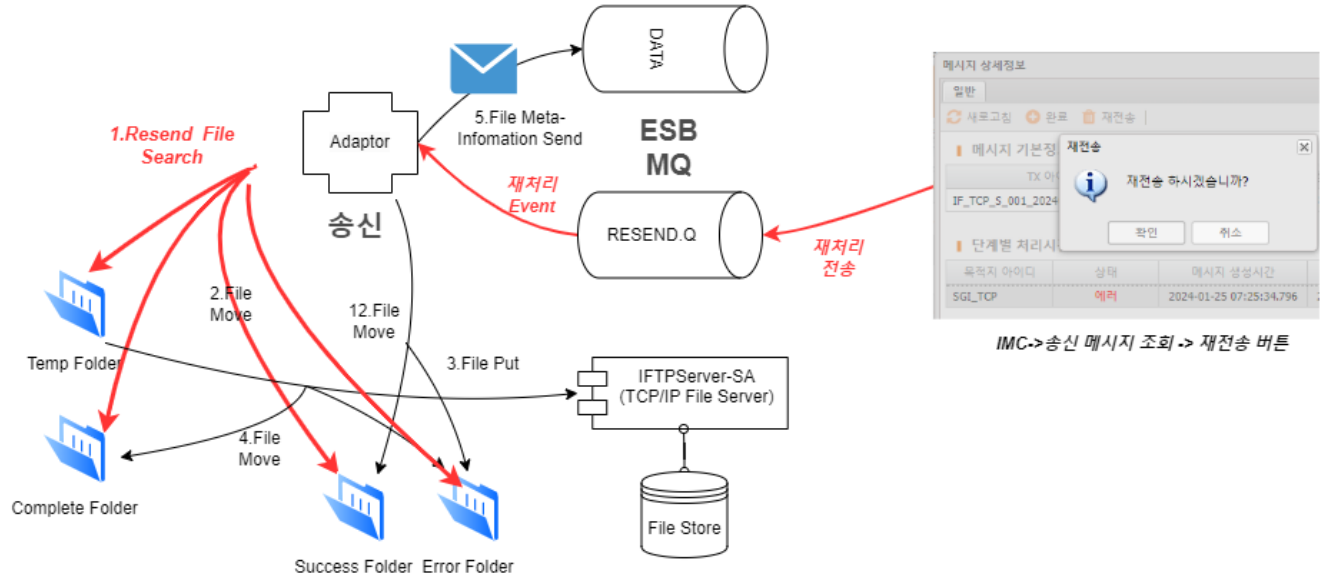
Table 29. spring.autoconfigure

Key	Value	Detail
exclude	ex) org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration	제외할 설정

8.5 재처리 기능

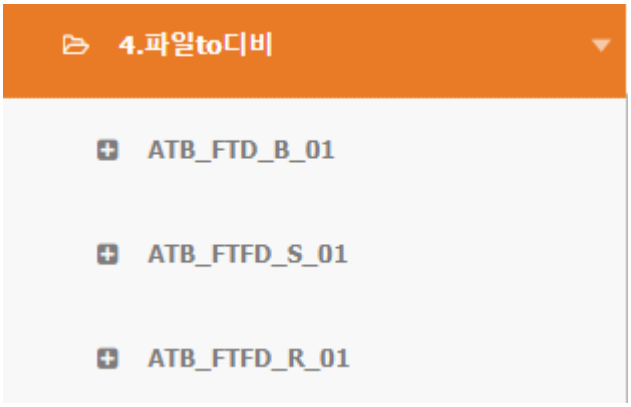
- IMC / 메시지 이력관리 / 송수신 메시지 조회 / 메시지 상세정보 / 실패한 목적지 아이디를 선택후 / 재전송 버튼을 클릭하면 재전송이 가능합니다.

File to File (재전송)



ℹ	재처리 기능은 송신 실패시 재처리할 파일이 temp/complete/success/error 폴더에 존재할 가능성을 염두해두고 각 폴더를 탐색하여 재송신 파일을 찾은후 지정하여 재송신을 합니다. 이후 발생되는 프로세스는 동일하게 동작합니다.

9 파일to디비

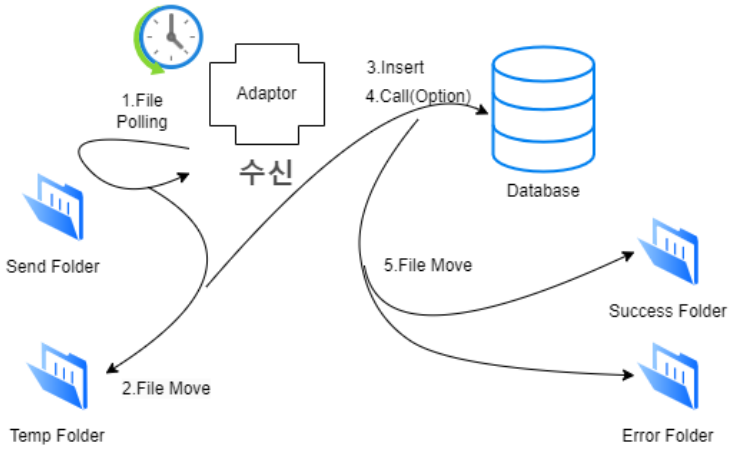


9.1 개요

FILE TO DB

9.2 Batch 방식 처리 구성도

File to DB (Batch)



	★ 송신 어댑터는 스케줄 이벤트에따라 1. 출발지(Source) 의 송신(Send) 폴더에 파일을 가져온후 2. 송신(처리)할 파일을 임시(Temp) 폴더로 이동시킨다. 3. 임시(Temp) 폴더로 이동시킨 파일을 대상(Target) DB 테이블에 INSERT를 한후 4. 옵션에 따라 프로시저를 호출 합니다. 5. 처리 결과에 따라 파일을 성공(Success) 또는 에러(Error) 폴더에 이동후 종료

	어뎁터의 서버에서 파일과 디비가 접근이 가능한상태에서 사용합니다.

9.3 ATB_FTD_B_01 (배치)

- 필수 공통 리소스

  DataSource

  데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: "FTD_BATCH_01"
    description: "FILE to DB Batch"
    prefix: FTD
    auto-config: false
    context:
      - interface-id: IF_FTD_003
        action-type: SENDER
        strategy: StandardFTDBatch
        cron-expression: 0/20 * * * * ?
        on-signal-loop: false
        procedure-call: true
        file-send-path: "/atb/send/IF_FTD_003"
        file-temp-path: "/atb/send/IF_FTD_003/temp"
        file-complete-path: "/atb/send/IF_FTD_003/complete"
        file-success-path: "/atb/send/IF_FTD_003/success"
        file-error-path: "/atb/send/IF_FTD_003/error"
        file-send-count: 1
        file-delete: true
        file-glob-pattern: "*.{csv,doc,xml,xlsx,zip,hwp,exe}"
        file-type: DELIMITER
        file-delimiter: ','
        file-delimiter-qualifier: '"'
        file-error-skip: false
        file-first-header: true
        send-system-code: A01
        receive-system-code: B01
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 30. atb.if-config

Key	Value	Detail
id	ex) FTD_BATCH_01	ATB 식별 ID
description	ex) "FILE to DB Batch"	연계 설명
prefix	ex) FTD	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 31. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_FTD_003	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardFTDBatch	전략 컴포넌트 ID
cronExpression	ex) 0/20 * * * * ?	스케줄 CRON 문법
file-send-path	ex) "/atb/send/IF_FTD_003"	송신 파일 경로
file-temp-path	ex) "/atb/send/IF_FTD_003/temp"	임시 파일 경로
file-complete-path	ex) "/atb/send/IF_FTD_003/complete"	완료 파일 경로
file-success-path	ex) "/atb/send/IF_FTD_003/success"	성공 파일 경로
file-error-path	ex) "/atb/send/IF_FTD_003/error"	에러 파일 경로
file-retry-count	ex) 3	파일 재전송 시도 횟수
file-send-count	ex) 1	파일 전송 갯수
file-delete	true, false	파일 삭제 여부
file-glob-pattern	ex) "*.{csv,doc,xml,xlsx,zip,hwp,exe}"	파일 Glob 패턴
file-type	ex) DELIMITER	파일의 타입
file-delimiter	ex) ','	Value 와 Value 사이의 구분자
file-delimiter-qualifier	ex) '"'	Value 를 감싸는 구분자
fileErrorSkip	true, false	에러 파일을 스킵할지 여부 (default : false)
file-first-header	true, false	파일의 헤더가 있는지 여부 (default : true)
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)

Table 32. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.

Key	Value	Detail
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 33. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true" />
        <setting name="useGeneratedKeys" value="true" />
        <setting name="defaultExecutorType" value="BATCH" />
        <setting name="defaultStatementTimeout" value="10000" />
    </settings>
    <mappers>
        <mapper resource="SQL_IF_FTD_003.xml" />
    </mappers>
</configuration>
```

Mybatis를 사용하기위한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_FTD_003.xml

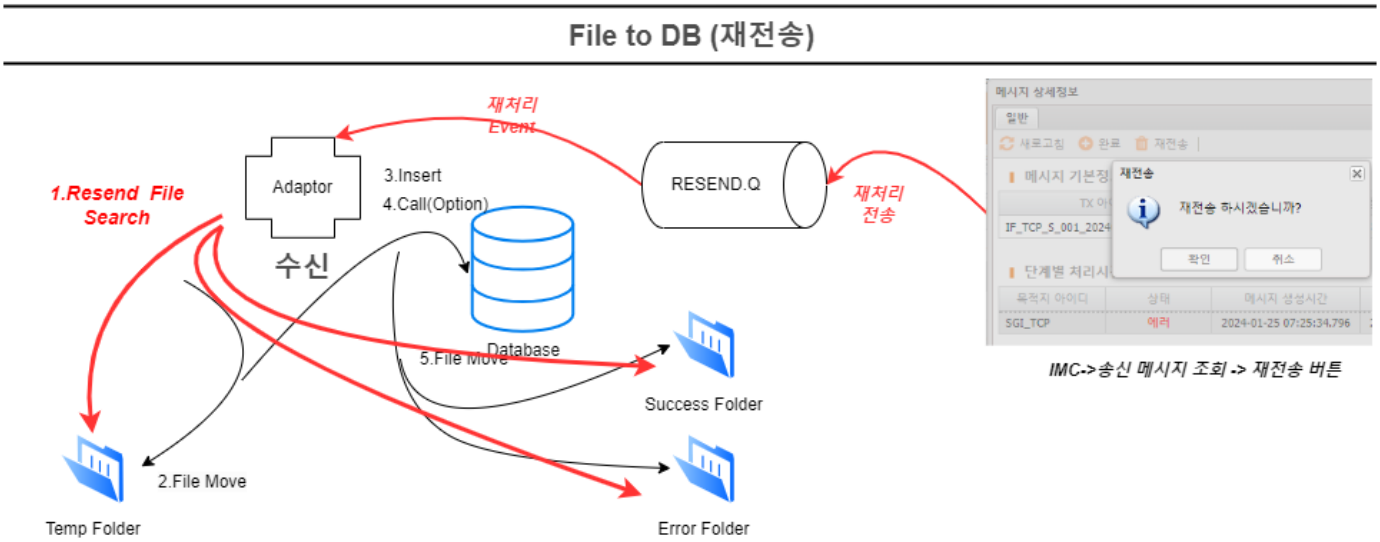
Mybatis 설정 파일

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_FTD_003">
    <insert id="INSERT" parameterType="hashmap">
        INSERT INTO IF_FTD_003 (         ESB_SEQ
                                             ,VARIABLE
                                             ,BREAKDOWN
                                             ,BREAKDOWN_CATEGORY
                                             ,YYEAR
                                             ,RD_VALUE
                                             ,STATUS
                                             ,FOOTNOTES
                                             ,UNIT_FOR_TSM_AND_CSV
                                             ,ESB_INTERFACE_ID
                                             ,ESB_STATUS_CD
                                             ,ESB_TRANSACTION_ID
                                             ,ESB_RECV_DT)
        values ( SEQ_IF_FTD_003.NEXTVAL ,
                 #{Variable     ,jdbcType=VARCHAR},
                 #{Breakdown     ,jdbcType=VARCHAR},
                 #{Breakdown_category     ,jdbcType=VARCHAR},
                 #{Year     ,jdbcType=VARCHAR},
                 #{RD_Value     ,jdbcType=VARCHAR},
                 #{Status     ,jdbcType=VARCHAR},
                 #{Footnotes     ,jdbcType=VARCHAR},
                 #{Unit_for_TSM_and_CSV     ,jdbcType=VARCHAR},
                 #{interfaceId     ,jdbcType=VARCHAR},
                 'N',
                 #{transactionId ,jdbcType=VARCHAR},
                 #{receiverMsgRecvDt     ,jdbcType=VARCHAR})
    </insert>

    <update id="CALL" statementType="CALLABLE">{CALL
SP_IF_FTD_003(#{transactionId ,jdbcType=VARCHAR})}</update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 INSERT , CALL 를 추가합니다.

9.4. 재처리 기능

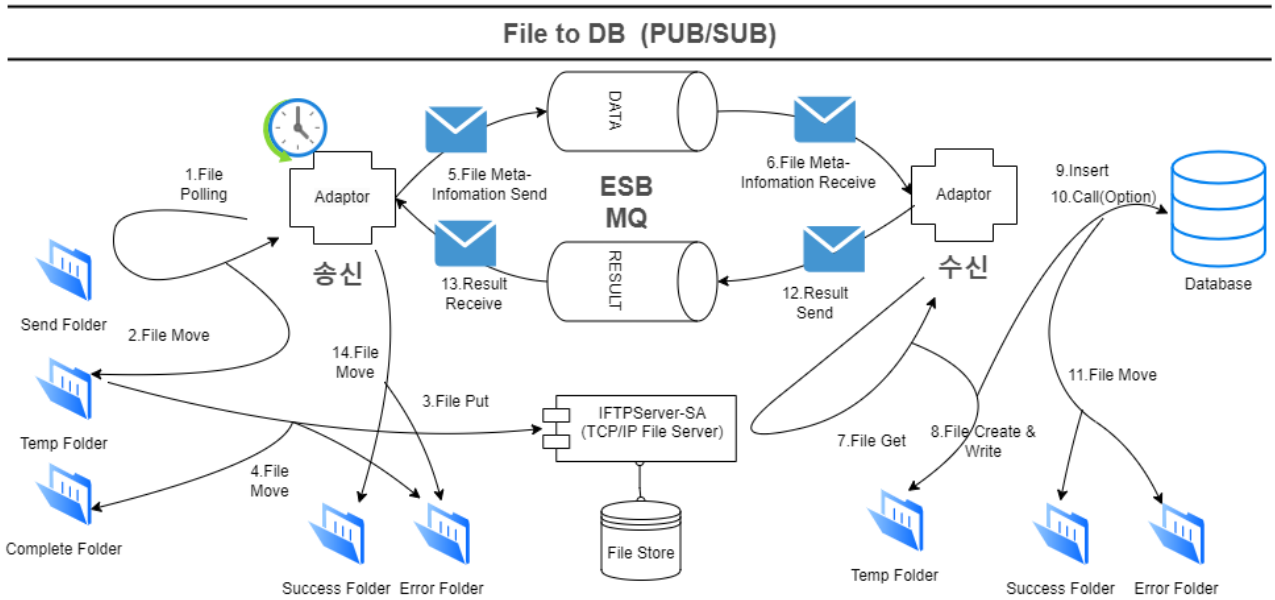
- IMC / 메시지 이력관리 / 송수신 메시지 조회 / 메시지 상세정보 / 실패한 목적지 아이디를 선택후 / 재전송 버튼을 클릭하면 재전송이 가능합니다.



재처리 기능은 송신 실패시 재처리할 파일이 temp/complete/success/error 폴더에 존재할 가능성을 염두해두고 각 폴더를 탐색하여 재송신 파일을 찾은후 지정하여 재송신을 합니다.

9.5 PUB/SUB 방식 처리 구성도

File Move



★송신 어댑터는 스케줄 이벤트에따라
1. 출발지(Source) 의 송신(Send) 폴더에 파일을 가져온후
2. 송신(처리)할 파일을 임시(Temp) 폴더로 이동시킨다.
3. 임시(Temp) 폴더에 있는 파일을 ESB Hub의 파일서버로 전송(PUT) 하고
4. 전송 실패하였다면 에러(Error) 폴더에 파일을 넣고 종료를 합니다. 정상적으로 송신하였다면.
5. 완료(Complete) 폴더 에 넣고 ESB MQ(MessageQueue) DATA 큐로 파일의 메타정보를 전송합니다.
★ 수신 어댑터는 ESB MQ(MessageQueue)인 DATA 큐로 부터
6. 파일의 메타정보를 수신하고 ESB Hub의 파일서버로 부터 임시(Temp) 폴더에 파일 수신(GET) 을 진행합니다.
7. 수신 받은 파일을 대상(Target) DB 테이블에 INSERT를 한후
8. 옵션에 따라 프로시저를 호출 합니다.
9. 처리 결과에 따라 파일을 성공(Success) 또는 에러(Error) 폴더에 이동시키고 처리 결과를 ESB MQ(MessageQueue)인 RESULT 큐로 전송합니다.
★ 송신 어댑터는
10. 처리결과를 ESB MQ(MessageQueue)로부터 수신하고
11. 처리결과에 따라 완료(Complete) 폴더에 있던 파일을 성공(Success) 또는 에러(Error) 폴더에 이동시킵니다.

9.6 ATB_FTFD_S_01 : PUB/SUB(송신)

- 필수 공통 리소스

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: "FTFD_SENDER_01"
    description: "(FILE SENDER) + JMS and Socket + (FILE + DB RECEIVER)"
    prefix: FTFD
    auto-config: false
    iftp:
      enabled: true
      data-packet-size: 4096
      retry-data-cnt: 100
      view-count: 1000
      transfer-sleep: 0
      encrypt: true
      remote-hosts:
        - clupine.duckdns.org:24052
```

```yaml
    context:
      - interface-id: IF_FTD_001
        action-type: SENDER
        strategy: StandardFTFDStrategy
        cron-expression: 0/20 * * * * ?
        on-signal-loop: false
        file-send-path: "/atb/send/IF_FTD_001"
        file-temp-path: "/atb/send/IF_FTD_001/temp"
        file-complete-path: "/atb/send/IF_FTD_001/complete"
        file-error-path: "/atb/send/IF_FTD_001/error"
        file-success-path: "/atb/send/IF_FTD_001/success"
        file-send-count: 10
        file-retry-count: 3
        file-delete: true
        file-glob-pattern: "*.{csv,doc,xml,xlsx,zip,hwp,exe}"
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: FTFD_RECEIVER_01
      - interface-id: IF_FTD_002
        action-type: SENDER
        strategy: StandardFTFDStrategy
        cron-expression: 0/20 * * * * ?
        on-signal-loop: false
        file-send-path: "/atb/send/IF_FTD_002"
        file-temp-path: "/atb/send/IF_FTD_002/temp"
        file-complete-path: "/atb/send/IF_FTD_002/complete"
        file-error-path: "/atb/send/IF_FTD_002/error"
        file-success-path: "/atb/send/IF_FTD_002/success"
        file-send-count: 10
        file-retry-count: 3
        file-delete: true
        file-glob-pattern: "*.{csv,doc,xml,xlsx,zip,hwp,exe}"
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: FTFD_RECEIVER_01
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
  autoconfigure:
    exclude:
      -
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

Table 34. atb.if-config

Key	Value	Detail
id	ex) "FTFD_SENDER_01"	ATB 식별 ID

Key	Value	Detail
description	ex) "(FILE SENDER) + JMS and Socket + (FILE + DB RECEIVER)"	연계 설명
prefix	ex) FTFD	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 35. atb.if-config.iftp

Key	Value	Detail
enabled	true, false	iftp 파일서버 클라이언트를 활성화 여부 (default : false)
data-packet-size	ex) 4096	전송 패킷 사이즈 (default : 4096)
retry-data-cnt	ex) 100	데이터 전송후 재시도 횟수 (default : 100)
view-count	ex) 1000	전송시 파일의 뷰 카운트 (default : 1000)
transfer-sleep	ex) 0	전송시간간격 (ms)
encrypt	true, false	파일암호화 여부 (default : false)
remote-hosts	ex) clupine.duckdns.org:24052	파일서버 호스트 (IP:PORT) 형식으로 입력

Table 36. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_FTD_001	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardFTFDStrategy	전략 컴포넌트 ID
cronExpression	ex) 0/20 * * * * ?	스케줄 CRON 문법
on-signal-loop	true, false	데이터가 존재할 경우 반복 여부 (default : false)
file-send-path	ex) "/atb/send/IF_FTD_001"	송신 파일 경로
file-temp-path	ex) "/atb/send/IF_FTD_001/temp"	임시 파일 경로

Key	Value	Detail
file-complete-path	ex) "/atb/send/IF_FTD_001/complete"	완료 파일 경로
file-error-path	ex) "/atb/send/IF_FTD_001/error"	에러 파일 경로
file-success-path	ex) "/atb/send/IF_FTD_001/success"	성공 파일 경로
file-send-count	ex) 10	파일 전송 갯수
file-retry-count	ex) 3	파일 재전송 시도 횟수
file-delete	true, false	파일 삭제 여부
file-glob-pattern	ex) "*.{csv,doc,xml,xlsx,zip,hwp,exe}"	파일 Glob 패턴
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) FTFD_RECEIVER_01	수신자 ATB 식별 ID
interfaceId	ex) IF_FTD_002	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardFTFDStrategy	전략 컴포넌트 ID
cronExpression	ex) 0/20 * * * * ?	스케줄 CRON 문법
on-signal-loop	true, false	데이터가 존재할 경우 반복 여부 (default : false)
file-send-path	ex) "/atb/send/IF_FTD_002"	송신 파일 경로
file-temp-path	ex) "/atb/send/IF_FTD_002/temp"	임시 파일 경로
file-complete-path	ex) "/atb/send/IF_FTD_002/complete"	완료 파일 경로
file-error-path	ex) "/atb/send/IF_FTD_002/error"	에러 파일 경로
file-success-path	ex) "/atb/send/IF_FTD_002/success"	성공 파일 경로
file-send-count	ex) 10	파일 전송 갯수
file-retry-count	ex) 3	파일 재전송 시도 횟수
file-delete	true, false	파일 삭제 여부

Key	Value	Detail
file-glob-pattern	ex) "*.{csv,doc,xml,xlsx,zip,hwp,exe}"	파일 Glob 패턴
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) FTFD_RECEIVER_01	수신자 ATB 식별 ID

Table 37. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 38. spring.autoconfigure

Key	Value	Detail
exclude	ex) org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration	제외할 설정

9.7 ATB_FTFD_R_01 : PUB/SUB(수신)

- 필수 공통 리소스

  DataSource

  데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```
atb:
```

```yaml
  if-config:
    id: "FTFD_RECEIVER_01"
    description: "(FILE SENDER) + JMS and Socket + (FILE + DB RECEIVER)"
    prefix: FTFD
    auto-config: false
    iftp:
      enabled: true
      data-packet-size: 4096
      retry-data-cnt: 100
      view-count: 1000
      transfer-sleep: 0
      encrypt: true
      remote-hosts:
        - clupine.duckdns.org:24052
    context:
      - interface-id: IF_FTD_001
        action-type: RECEIVER
        strategy: StandardFTFDStrategy
        procedure-call: true
        file-recv-path: "/atb/recv/IF_FTD_001/"
        file-temp-path: "/atb/recv/IF_FTD_001/temp"
        file-success-path: "/atb/recv/IF_FTD_001/success"
        file-error-path: "/atb/recv/IF_FTD_001/error"
        file-retry-count: 3
        file-type: DELIMITER
        file-delimiter: ','
        file-delimiter-qualifier: '"'
        file-first-header: true
        file-error-skip: false
      - interface-id: IF_FTD_002
        action-type: RECEIVER
        strategy: StandardFTFDStrategy
        procedure-call: true
        file-recv-path: "/atb/recv/IF_FTD_002/"
        file-temp-path: "/atb/recv/IF_FTD_002/temp"
        file-success-path: "/atb/recv/IF_FTD_002/success"
        file-error-path: "/atb/recv/IF_FTD_002/error"
        file-retry-count: 3
        file-type: DELIMITER
        file-delimiter: ','
        file-delimiter-qualifier: '"'
        file-first-header: false
        file-error-skip: true
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 39. atb.if-config

Key	Value	Detail
id	ex) "FTFD_RECEIVER_01"	ATB 식별 ID
description	ex) "(FILE SENDER) + JMS and Socket + (FILE + DB RECEIVER)"	연계 설명
prefix	ex) FTFD	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 40. atb.if-config.iftp

Key	Value	Detail
enabled	true, false	iftp 파일서버 클라이언트를 활성화 여부 (default : false)
data-packet-size	ex) 4096	전송 패킷 사이즈 (default : 4096)
retry-data-cnt	ex) 100	데이터 전송후 재시도 횟수 (default : 100)
view-count	ex) 1000	전송시 파일의 뷰 카운트 (default : 1000)
transfer-sleep	ex) 0	전송시간간격 (ms)
encrypt	true, false	파일암호화 여부 (default : false)
remote-hosts	ex) clupine.duckdns.org:24052	파일서버 호스트 (IP:PORT) 형식으로 입력

Table 41. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_FTD_001	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardFTFDStrategy	전략 컴포넌트 ID
file-recv-path	ex) "/atb/recv/IF_FTD_001"	수신 파일 경로
file-temp-path	ex) "/atb/recv/IF_FTD_001/temp"	임시 파일 경로
file-success-path	ex) "/atb/recv/IF_FTD_001/success"	성공 파일 경로
file-error-path	ex) "/atb/recv/IF_FTD_001/error"	에러 파일 경로

Key	Value	Detail
file-retry-count	ex) 3	파일 재전송 시도 횟수
file-type	ex) DELIMITER	파일의 타입
file-delimiter	ex) ','	Value 와 Value 사이의 구분자
file-delimiter-qualifier	ex) '"'	Value 를 감싸는 구분자
file-first-header	true, false	파일의 헤더가 있는지 여부 (default : true)
fileErrorSkip	true, false	에러 파일을 스킵할지 여부 (default : false)
interfaceId	ex) IF_FTD_002	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardFTFDStrategy	전략 컴포넌트 ID
file-recv-path	ex) "/atb/recv/IF_FTD_002"	수신 파일 경로
file-temp-path	ex) "/atb/recv/IF_FTD_002/temp"	임시 파일 경로
file-success-path	ex) "/atb/recv/IF_FTD_002/success"	성공 파일 경로
file-error-path	ex) "/atb/recv/IF_FTD_002/error"	에러 파일 경로
file-retry-count	ex) 3	파일 재전송 시도 횟수
file-type	ex) DELIMITER	파일의 타입
file-delimiter	ex) ','	Value 와 Value 사이의 구분자
file-delimiter-qualifier	ex) '"'	Value 를 감싸는 구분자
file-first-header	true, false	파일의 헤더가 있는지 여부 (default : true)
fileErrorSkip	true, false	에러 파일을 스킵할지 여부 (default : false)

Table 42. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.

Key	Value	Detail
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 43. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true" />
        <setting name="useGeneratedKeys" value="true" />
        <setting name="defaultExecutorType" value="BATCH" />
        <setting name="defaultStatementTimeout" value="10000" />
    </settings>
    <mappers>
        <mapper resource="SQL_IF_FTD_001.xml" />
        <mapper resource="SQL_IF_FTD_002.xml" />
    </mappers>
</configuration>
```

Mybatis를 사용하기위한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_FTD_001.xml

Mybatis 설정 파일

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_FTD_001">
    <insert id="INSERT" parameterType="hashmap">
        INSERT INTO IF_FTD_001 (         ESB_SEQ
                                             ,VARIABLE
                                             ,BREAKDOWN
                                             ,BREAKDOWN_CATEGORY
                                             ,YYEAR
                                             ,RD_VALUE
                                             ,STATUS
                                             ,FOOTNOTES
                                             ,UNIT_FOR_TSM_AND_CSV
                                             ,ESB_INTERFACE_ID
                                             ,ESB_STATUS_CD
                                             ,ESB_TRANSACTION_ID
                                             ,ESB_RECV_DT)
        values ( SEQ_IF_FTD_001.NEXTVAL ,
                 #{Variable     ,jdbcType=VARCHAR},
                 #{Breakdown     ,jdbcType=VARCHAR},
                 #{Breakdown_category     ,jdbcType=VARCHAR},
                 #{Year     ,jdbcType=VARCHAR},
                 #{RD_Value     ,jdbcType=VARCHAR},
                 #{Status     ,jdbcType=VARCHAR},
                 #{Footnotes     ,jdbcType=VARCHAR},
                 #{Unit_for_TSM_and_CSV     ,jdbcType=VARCHAR},
                 #{interfaceId     ,jdbcType=VARCHAR},
                 'N',
                 #{transactionId ,jdbcType=VARCHAR},
                 #{receiverMsgRecvDt     ,jdbcType=VARCHAR})
    </insert>

    <update id="CALL" statementType="CALLABLE">{CALL
SP_IF_FTD_001(#{transactionId ,jdbcType=VARCHAR})}</update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 INSERT , CALL 를 추가합니다.

SQL_IF_FTD_002.xml

Mybatis 설정 파일

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_FTD_002">
    <insert id="INSERT" parameterType="hashmap">
        INSERT INTO IF_FTD_002 (         ESB_SEQ
                                             ,VARIABLE
                                             ,BREAKDOWN
                                             ,BREAKDOWN_CATEGORY
                                             ,YYEAR
                                             ,RD_VALUE
                                             ,STATUS
                                             ,FOOTNOTES
                                             ,UNIT_FOR_TSM_AND_CSV
                                             ,ESB_INTERFACE_ID
                                             ,ESB_STATUS_CD
                                             ,ESB_TRANSACTION_ID
                                             ,ESB_RECV_DT)
        values ( SEQ_IF_FTD_002.NEXTVAL ,
                 #{Variable     ,jdbcType=VARCHAR},
                 #{Breakdown     ,jdbcType=VARCHAR},
                 #{Breakdown_category     ,jdbcType=VARCHAR},
                 #{Year     ,jdbcType=VARCHAR},
                 #{RD_Value     ,jdbcType=VARCHAR},
                 #{Status     ,jdbcType=VARCHAR},
                 #{Footnotes     ,jdbcType=VARCHAR},
                 #{Unit_for_TSM_and_CSV     ,jdbcType=VARCHAR},
                 #{interfaceId     ,jdbcType=VARCHAR},
                 'N',
                 #{transactionId ,jdbcType=VARCHAR},
                 #{receiverMsgRecvDt     ,jdbcType=VARCHAR})
    </insert>

    <update id="CALL" statementType="CALLABLE">{CALL
SP_IF_FTD_002(#{transactionId ,jdbcType=VARCHAR})}</update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 INSERT , CALL 를 추가합니다.

IF_FTD_002.pzmap

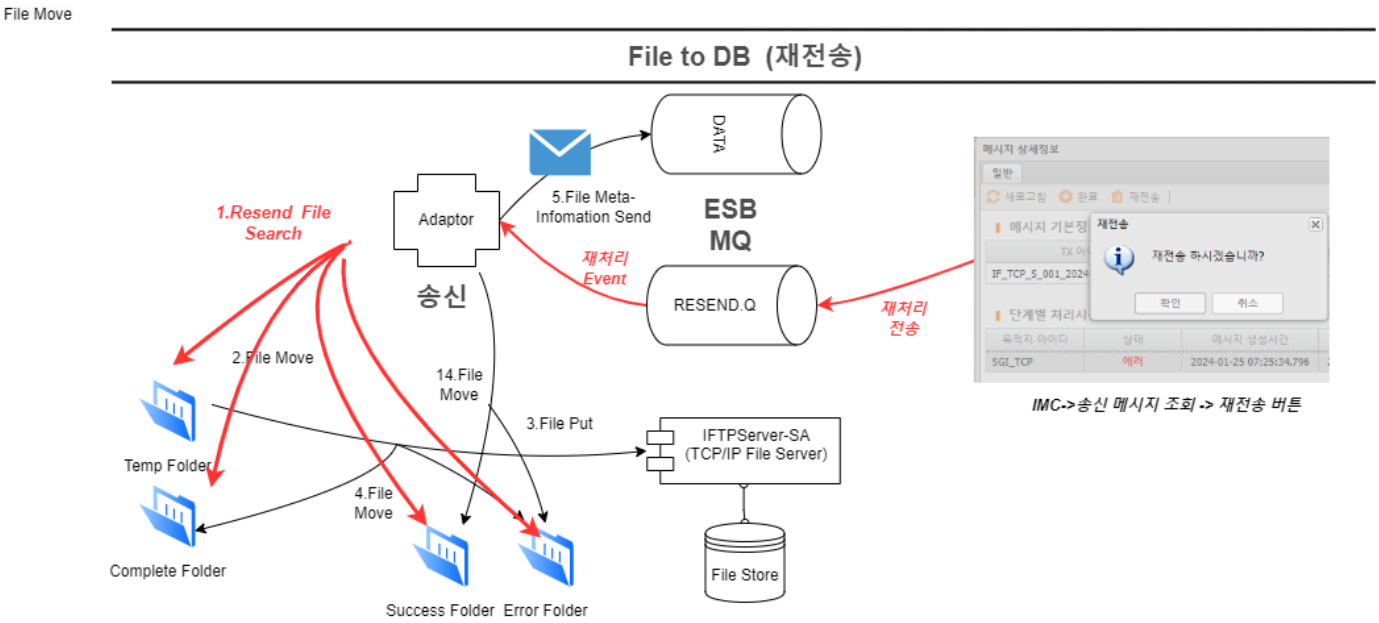
Mapping 파일

```xml
<?xml version="1.0"?>
<!-- DTD can be pulled from the Jar or over the web-->
<!DOCTYPE PZMAP SYSTEM  "flatpack.dtd" >
<!--<!DOCTYPE PZMAP SYSTEM  "http://flatpack.sourceforge.net/flatpack.dtd"
>-->
<PZMAP>
    <COLUMN name="Variable" />
    <COLUMN name="Breakdown" />
    <COLUMN name="Breakdown_category" />
    <COLUMN name="Year" />
    <COLUMN name="RD_Value" />
    <COLUMN name="Status" />
    <COLUMN name="Footnotes" />
    <COLUMN name="Unit_for_TSM_and_CSV" />
</PZMAP>
```

파일에 헤더정보가 없는경우는 인터페이스ID.pzmap 파일을 생성하고 맵핑할 필드명을 순서대로 입력합니다.

9.8 재처리 기능

- IMC / 메시지 이력관리 / 송수신 메시지 조회 / 메시지 상세정보 / 실패한 목적지 아이디를 선택후 / 재전송 버튼을 클릭하면 재전송이 가능합니다.



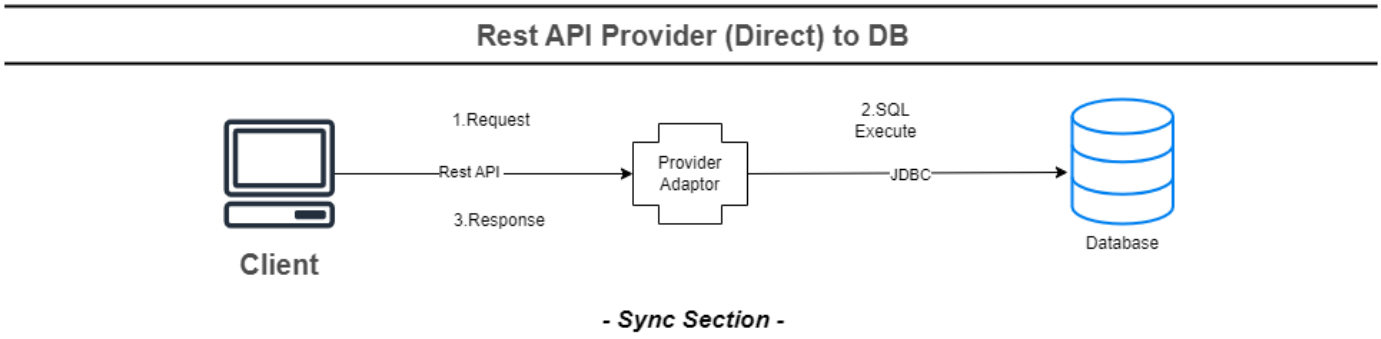
재처리 기능은 송신 실패시 재처리할 파일이 temp/complete/success/error 폴더에 존재할 가능성을 염두해두고 각 폴더를 탐색하여 재송신 파일을 찾은후 지정하여 재송신을 합니다.

10 REST API 프로바이더 (Direct)

10.1 개요

REST API 프로바이더 (DIRECT)

10.2 DIRECT DB 처리 구성도



	★ 어댑터는 Rest API Server 가 되어 1. 이용자(Client)로 부터 요청(Request)이 들어오면 2. 어댑터는 인터페이스에 따른 SQL문을 실행하여 SELECT,UPDATE,INSERT,DELETE,Procedure Call 를 수행합니다. 3. 처리한 결과를 응답(Response)합니다.

10.3 ATB_REST_PRV_DIRECT_DB : DIRECT DB

- 필수 공통 리소스

  DataSource

  데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

application.yml

어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: REST_PROVIDER_DB_01
    description: "REST PROVIDER (Direct DB)"
    prefix: REST_PRV
    route-type: NONE
    auto-config: false
    context:
      - interfaceId: IF_REST_DB_01
        actionType: CUSTOMIZE
        strategy: StandardDBInsertStrategy
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_PROVIDER_DB_01
      - interfaceId: IF_REST_DB_02
        actionType: CUSTOMIZE
        strategy: StandardDBSelectStrategy
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_PROVIDER_DB_01
      - interfaceId: IF_REST_DB_03
        actionType: CUSTOMIZE
        strategy: StandardDBUpdateStrategy
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_PROVIDER_DB_01
      - interfaceId: IF_REST_DB_04
        actionType: CUSTOMIZE
        strategy: StandardDBDeleteStrategy
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_PROVIDER_DB_01
server:
  port: 24219
spring:
  main:
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 44. atb.if-config

Key	Value	Detail
id	ex) REST_PROVIDER_DB_01	ATB 식별 ID
description	ex) "REST PROVIDER (Direct DB)"	연계 설명
prefix	ex) REST_PRV	식별자
route-type	ex) NONE	라우팅 타입 (NONE: 자체처리 , JMS: REMOTE Adaptor로 전송)
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 45. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_REST_DB_01	인터페이스 ID
actionType	CUSTOMIZE	액션 타입
strategy	ex) StandardDBInsertStrategy	전략 컴포넌트 ID
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) REST_PROVIDER_DB_01	수신자 ATB 식별 ID

Table 46. server

Key	Value	Detail
port	ex) 24219	HTTP(HTTPS) 서비스 포트

Table 47. spring.main

Key	Value	Detail
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 48. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true" />
        <setting name="useGeneratedKeys" value="true" />
        <setting name="defaultExecutorType" value="BATCH" />
        <setting name="defaultStatementTimeout" value="10000" />
    </settings>
    <mappers>
      <mapper resource="SQL_IF_REST_DB_01.xml" />
      <mapper resource="SQL_IF_REST_DB_02.xml" />
      <mapper resource="SQL_IF_REST_DB_03.xml" />
      <mapper resource="SQL_IF_REST_DB_04.xml" />
    </mappers>
</configuration>
```

Mybatis를 사용하기위한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_REST_DB_01.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_01">
   <insert id="INSERT" parameterType="hashmap">
        INSERT INTO COMPANY_DATA(ID, NAME, COMPANY_NAME, APP_NAME,
DOMAIN_NAME)
        values (
                 SEQ_COMPANY_DATA.NEXTVAL ,
                 #{name     ,jdbcType=VARCHAR},
                 #{companyName     ,jdbcType=VARCHAR},
                 #{appName     ,jdbcType=VARCHAR},
                 #{domainName     ,jdbcType=VARCHAR}
                 )
    </insert>
</mapper>
```

namespace는 인터페이스 ID를 넣고 INSERT 를 추가합니다.

SQL_IF_REST_DB_02.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_02">
   <select id="SELECT" parameterType="hashmap" resultType="hashmap">
        SELECT * FROM COMPANY_DATA
        WHERE NAME LIKE '%'||#{name}||'%'
    </select>
</mapper>
```

namespace는 인터페이스 ID를 넣고 SELECT 를 추가합니다.

SQL_IF_REST_DB_03.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_03">
    <update id="UPDATE" parameterType="hashmap">
        UPDATE COMPANY_DATA
        SET
        <trim suffixOverrides=",">
           <if test="companyName != null and companyName != ''"
>COMPANY_NAME =  #{companyName,jdbcType=VARCHAR} ,</if>
           <if test="appName != null and appName != ''" >APP_NAME =
#{appName,jdbcType=VARCHAR}, </if>
           <if test="domainName != null and domainName != ''" >DOMAIN_NAME
= #{domainName,jdbcType=VARCHAR}, </if>
           <if test="name != null and name != ''" >NAME =
#{name,jdbcType=VARCHAR}, </if>
        </trim>
        WHERE ID = #{id}
    </update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 UPDATE 를 추가합니다.

SQL_IF_REST_DB_04.xml

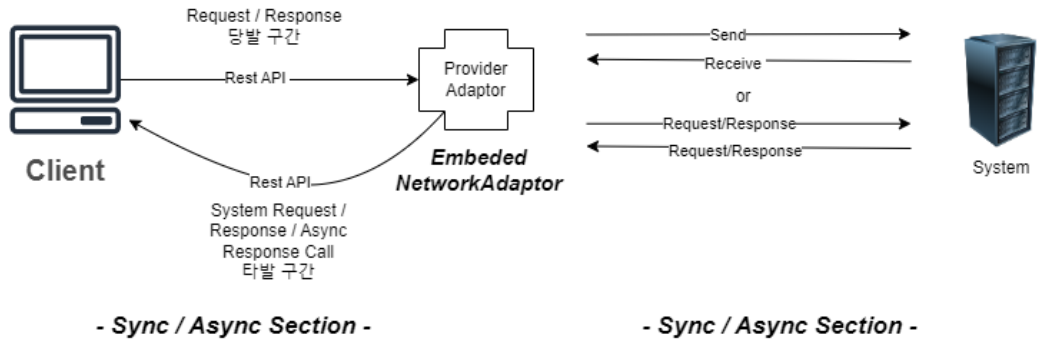
Mybatis 설정 파일

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_04">
    <update id="DELETE" parameterType="hashmap">
        DELETE COMPANY_DATA
        WHERE ID = #{id}
    </update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 DELETE 를 추가합니다.

10.4 DIRECT TCP/IP 처리 구성도

Rest API Provider (Direct) to TCP/IP(Embeded NetworkAdaptor)



★ 어댑터는 Rest API Server 가 되어
1. 이용자(Client)로 부터 요청(Request)이 들어오면
2. 내장되어있는 TCP/IP 통신 모듈 NetworkAdaptor를 통해 대상(Target) 시스템에 전문을 전달(Send Or Request)합니다.
3. 대상(Target) 시스템으로부터 전문을 수신(Receive)받으면 요청한 API로 응답을 하거나 비동기로 Client측에 호출을 하여 전달합니다.

진행 순서가 없는건 단방향이 아닌 Client에서 시작이되는 당발과 System에서 시작이 되는 타발로 양방향 통신을 구성하기 때문입니다.

10.5 ATB_REST_PRV_DIRECT_NA : DIRECT TCP/IP

- 필수 공통 리소스

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

application.yml

어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: RSTNA_E
    description: "REST SERVER/CLIENT CONVERSION DUPLEX NETWORK
ADAPTOR(EMBEDDED)"
    embedded-network-adaptor:
        type: LENGTH
        tcp-mode: SYNC
        remote-hosts:
          - 127.0.0.1
        remote-ports:
          - 8004
        max-frame-length: 4096
        length-field-offset: 0
        length-field-length: 4
        length-adjustment: 0
        initial-bytes-to-strip: 0
        string-value-use: true
        message-id: 39
        message-id-len: 20
        work-code: 24
        work-code-len: 7
        connect-timeout-sec: 30
        reconnect-sec: 30
        recv-task-count: 10
        hex-mode: false
        call-type: JMS
        src-org: SHE
        trg-org: SCI
    if-result:
        inject: true
        tcp-plain-data: true
    auto-config: false
    context:
      - interfaceId: IF_TCP_S_001
        actionType: CUSTOMIZE
        strategy: StandardNetworkAdaptorStrategy
        tcpMode: SYNC
        out-mapper-type: PARSER
        in-mapper-type: PARSER
        parser-id: SHineSCIParser
        send-system-code: A01
        receive-system-code: SCI
        receiver-ids: SGI_TCP
server:
```

```
  port: 24220
spring:
  autoconfigure:
    exclude:
      -
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
  main:
    allow-bean-definition-overriding: true
```

Table 49. atb.if-config

Key	Value	Detail
id	ex) RSTNA_E	ATB 식별 ID
description	ex) "REST SERVER/CLIENT CONVERSION DUPLEX NETWORK ADAPTOR(EMBEDDED)"	연계 설명
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 50. atb.if-config.embedded-network-adaptor

Key	Value	Detail
type	ex) LENGTH	네트워크 어댑터 타입 (LENGTH, DELIMITER, FIXED)
tcp-mode	ex) SYNC	TCP/IP 통신 모드 (SYNC, ASYNC)
remote-hosts	ex) 127.0.0.1	대상 시스템 IP
remote-ports	ex) 8004	대상 시스템 PORT
max-frame-length	ex) 4096	최대 프레임 길이
length-field-offset	ex) 0	길이 필드 오프셋
length-field-length	ex) 4	길이 필드 길이
length-adjustment	ex) 0	길이 조정
initial-bytes-to-strip	ex) 0	초기 바이트 제거
string-value-use	true, false	문자열 사용 여부
message-id	ex) 39	메시지 ID(요청과 응답의 메시지가 짝이 되어야할 고유한값의 index) ex)전문거래번호
message-id-len	ex) 20	메시지 ID 길이

Key	Value	Detail
work-code	ex) 24	업무를 식별할수있는 업무구분코드 인덱스
work-code-len	ex) 7	업무구분코드 길이
connect-timeout-sec	ex) 30	연결 타임아웃 시간
reconnect-sec	ex) 30	재연결 시간
recv-task-count	ex) 10	수신 작업자 갯수
hex-mode	true, false	16진수 모드 여부 (default : false)
call-type	ex) JMS	호출 타입 (JMS, HTTP)
src-org	ex) SHE	송신기관(로그 이력용)
trg-org	ex) SCI	수신기관(로그 이력용)

Table 51. atb.if-config.if-result

Key	Value	Detail
inject	true, false	Provider의 응답 메시지에 연계 결과을 주입할지 여부 (default : false)
tcp-plain-fata	true, false	TCP/IP 통신시 평문 데이터를 주입할지 여부 (default : false)

Table 52. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_TCP_S_001	인터페이스 ID
actionType	CUSTOMIZE	액션 타입 (CUSTOMIZE 고정)
strategy	ex) StandardNetworkAdaptorStrategy	전략 컴포넌트 ID
tcpMode	ex) SYNC	network-adaptor 설정과는 별개로 해당 인터페이스를 동기로 처리할지 비동기로 처리할지에 대한 설정
out-mapper-type	ex) PARSER	송신 메시지 맵핑 타입 (MAPPER, PARSER)
in-mapper-type	ex) PARSER	수신 메시지 맵핑 타입 (MAPPER, PARSER)
parser-id	ex) SHineSCIParser	PASER 타입일경우 사용할 파서 컴포넌트 ID

Key	Value	Detail
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) SGI_TCP	수신자 ATB 식별 ID

Table 53. server

Key	Value	Detail
port	ex) 24220	HTTP(HTTPS) 서비스 포트

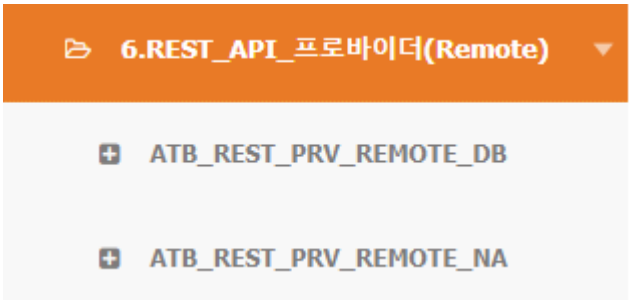
Table 54. spring.autoconfigure

Key	Value	Detail
exclude	ex) org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration	제외할 설정

Table 55. spring.main

Key	Value	Detail
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

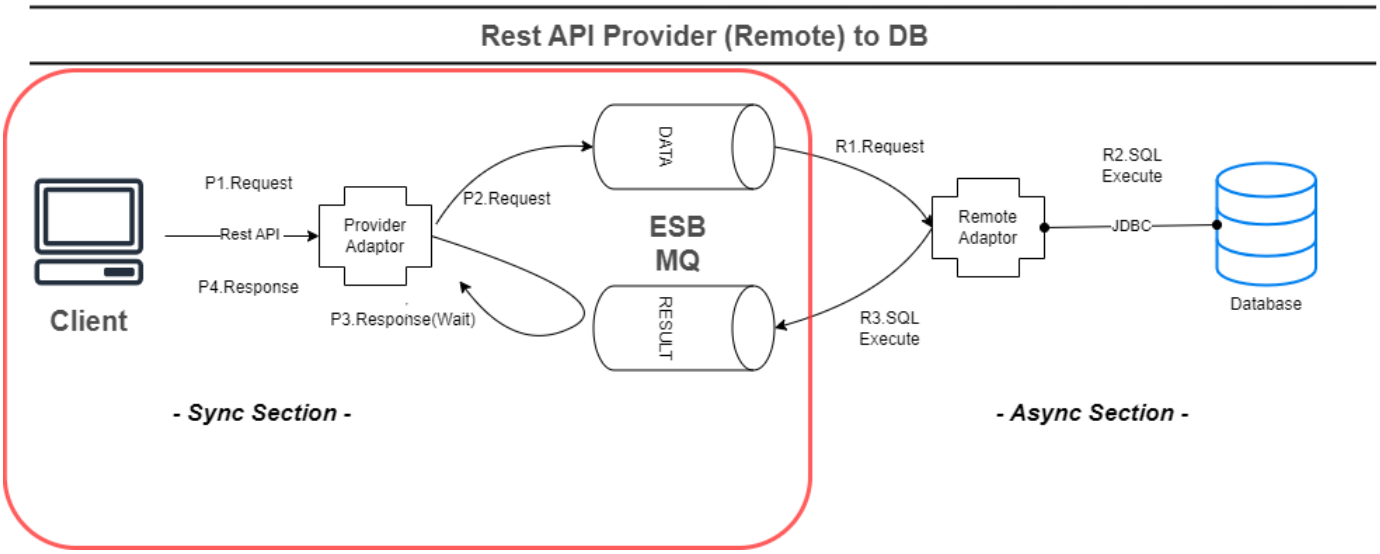
11 REST API 프로바이더 (Remote)



11.1 개요

REST API 프로바이더 (REMOTE)

11.2 REMOTE DB 처리 구성도 (좌측 Provider Adaptor)



	★ 어댑터는 Rest API Server 가 되어 1. 이용자(Client)로 부터 요청(Request)이 들어오면 2. 요청 메시지를 ESB MQ를 통하여 요청하고 3. 응답 메시지를 ESB MQ를 통하여 응답받습니다.

	REST API 프로바이더 DIRECT DB 방식에서 Provider와 DB가 분리되어 위의 그림처럼 좌측 영역만 수행을 합니다.

11.3 ATB_REST_PRV_REMOTE_DB : Provider

- 필수 공통 리소스

  DataSource

  데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: REST_PROVIDER_DB_02
    description: "REST PROVIDER (Remote DB)"
    prefix: REST_PRV
    route-type: JMS
    auto-config: false
    context:
      - interfaceId: IF_REST_DB_01
        actionType: CUSTOMIZE
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_REMOTE_DB_01
      - interfaceId: IF_REST_DB_02
        actionType: CUSTOMIZE
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_REMOTE_DB_01
      - interfaceId: IF_REST_DB_03
        actionType: CUSTOMIZE
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_REMOTE_DB_01
      - interfaceId: IF_REST_DB_04
        actionType: CUSTOMIZE
        send-system-code: A01
        receive-system-code: B01
        receiver-ids: REST_REMOTE_DB_01
server:
  port: 24219
spring:
  main:
    allow-bean-definition-overriding: true
```

Table 56. atb.if-config

Key	Value	Detail
id	ex) REST_PROVIDER_DB_02	ATB 식별 ID
description	ex) "REST PROVIDER (Remote DB)"	연계 설명
prefix	ex) REST_PRV	식별자
route-type	ex) JMS	라우팅 타입 (NONE: 자체처리 , JMS: REMOTE Adaptor로 전송)

Key	Value	Detail
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 57. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_REST_DB_01	인터페이스 ID
actionType	CUSTOMIZE	액션 타입
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) REST_REMOTE_DB_01	수신자 ATB 식별 ID

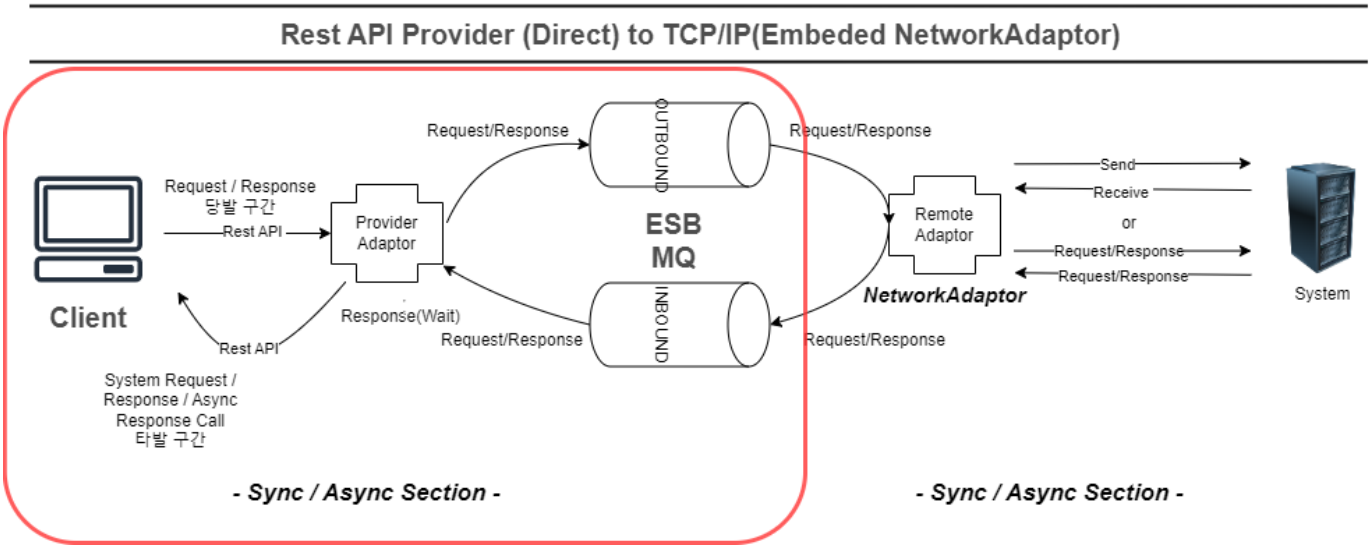
Table 58. server

Key	Value	Detail
port	ex) 24219	HTTP(HTTPS) 서비스 포트

Table 59. spring.main

Key	Value	Detail
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

11.4 REMOTE TCP/IP 처리 구성도 (좌측 Provider Adaptor)



	★ 어댑터는 Rest API Server 가 되어 1. 이용자(Client)로 부터 요청(Request)이 들어오면 2. 요청 메시지를 ESB MQ를 통하여 요청하고 3. 응답 메시지를 ESB MQ를 통하여 응답받습니다.

	REST API 프로바이더 DIRECT DB 방식에서 Provider와 DB가 분리되어 위의 그림처럼 좌측 영역만 수행을 합니다.

11.5 ATB_REST_PRV_REMOTE_NA : Provider

- 필수 공통 리소스

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: RSTNA_R
    description: "REST SERVER/CLIENT CONVERSION DUPLEX NETWORK
ADAPTOR(REMOTE)"
    remote-network-adaptor:
        remote-queue-map:
          SGI_TCP: OUT.SGI_TCP
        inbound-queue: IN.SGI_TCP
        receive-timeout: 12000
    auto-config: false
    context:
      - interfaceId: IF_TCP_S_001
        actionType: CUSTOMIZE
        strategy: StandardNetworkAdaptorStrategy
        tcpMode: SYNC
        out-mapper-type: PARSER
        in-mapper-type: PARSER
        parser-id: SHineSCIParser
        send-system-code: A01
        receive-system-code: SCI
        receiver-ids: SGI_TCP
server:
  port: 24220
spring:
  autoconfigure:
    exclude:
      -
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
  main:
    allow-bean-definition-overriding: true
```

Table 60. atb.if-config

Key	Value	Detail
id	ex) RSTNA_R	ATB 식별 ID
description	ex) "REST SERVER/CLIENT CONVERSION DUPLEX NETWORK ADAPTOR(REMOTE)"	연계 설명
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 61. atb.if-config.remote-network-adaptor.remote-queue-map

Key	Value	Detail
SGI_TCP	OUT>SGI_TCP	수신자 ATB 식별 ID

Table 62. atb.if-config.remote-network-adaptor

Key	Value	Detail
inbound-queue	IN.SGI_TCP	수신자 ATB 식별 ID
receive-timeout	12000	수신 타임아웃 시간

Table 63. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_TCP_S_001	인터페이스 ID
actionType	CUSTOMIZE	액션 타입
strategy	ex) StandardNetworkAdaptorStrategy	전략 컴포넌트 ID
tcpMode	ex) SYNC	network-adaptor 설정과는 별개로 해당 인터페이스를 동기로 처리할지 비동기로 처리할지에 대한 설정
out-mapper-type	ex) PARSER	송신 메시지 맵핑 타입 (MAPPER, PARSER)
in-mapper-type	ex) PARSER	수신 메시지 맵핑 타입 (MAPPER, PARSER)
parser-id	ex) SHineSCIParser	PASER 타입일경우 사용할 파서 컴포넌트 ID
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)
receiver-ids	ex) SGI_TCP	수신자 ATB 식별 ID

Table 64. server

Key	Value	Detail
port	ex) 24220	HTTP(HTTPS) 서비스 포트

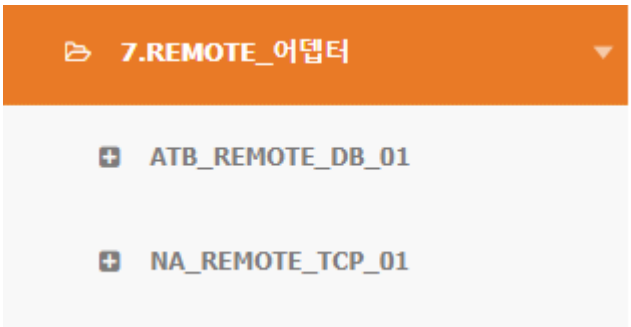
Table 65. spring.autoconfigure

Key	Value	Detail
exclude	ex) org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration	제외할 설정

Table 66. spring.main

Key	Value	Detail
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

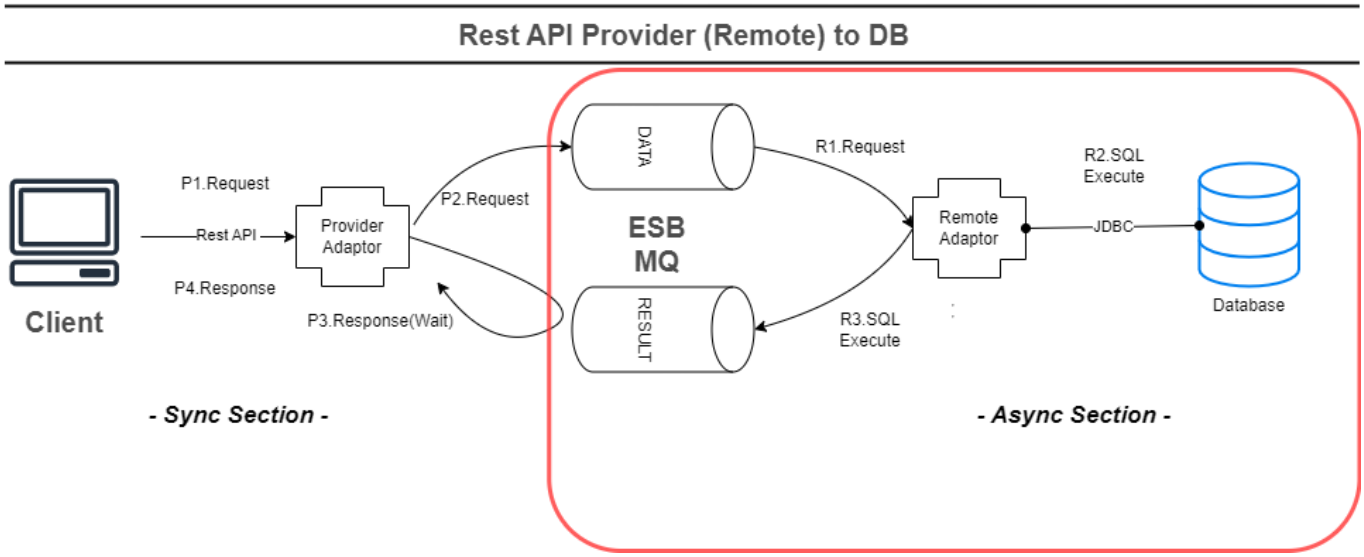
12 REMOTE 어댑터



12.1 개요

REMOTE 어댑터

12.2 REMOTE DB 처리 구성도(우측 Remote Adaptor)



	★ Remote 어댑터는 ESB MQ로 부터 1. 요청 메시지를 받아 인터페이스에 따른 SQL문을 실행하여 SELECT,UPDATE,INSERT,DELETE,Procedure Call 를 수행합니다. 2. 처리한 결과를 응답 메시지를 ESB MQ로 전달합니다.

	REST API 프로바이더 DIRECT DB 방식에서 Provider와 DB가 분리되어 위의 그림처럼 우측 영역만 수행을 합니다.

12.3 ATB_REMOTE_DB_01 : REMOTE DB

- 필수 공통 리소스

  DataSource

  데이터베이스 연결을 위한 DataSource , id를 dataSource로 등록하면 자동 Injection

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
atb:
  if-config:
    id: REST_REMOTE_DB_01
    description: "REST Remote DB"
    prefix: REST_PRV
    auto-config: false
    context:
      - interfaceId: IF_REST_DB_01
        actionType: RECEIVER
        strategy: StandardDBInsertStrategy
        send-system-code: A01
        receive-system-code: B01
      - interfaceId: IF_REST_DB_02
        actionType: RECEIVER
        strategy: StandardDBSelectStrategy
        send-system-code: A01
        receive-system-code: B01
      - interfaceId: IF_REST_DB_03
        actionType: RECEIVER
        strategy: StandardDBUpdateStrategy
        send-system-code: A01
        receive-system-code: B01
      - interfaceId: IF_REST_DB_04
        actionType: RECEIVER
        strategy: StandardDBDeleteStrategy
        send-system-code: A01
        receive-system-code: B01
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

Table 67. atb.if-config

Key	Value	Detail
id	ex) REST_REMOTE_DB_01	ATB 식별 ID
description	ex) "REST Remote DB"	연계 설명
prefix	ex) REST_PRV	식별자
auto-config	ex) true	인터페이스 관리 설정에서 가져옵니다. (default : false) *v1.0.2부터 지원*

Table 68. atb.if-config.context

Key	Value	Detail
interfaceId	ex) IF_REST_DB_01	인터페이스 ID
actionType	SENDER, RECEIVER	액션 타입
strategy	ex) StandardDBInsertStrategy	전략 컴포넌트 ID
send-system-code	ex) A01	송신 시스템 코드(이력용)
receive-system-code	ex) B01	수신 시스템 코드(이력용)

Table 69. spring.main

Key	Value	Detail
web-application-type	ex) none	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 70. mybatis

Key	Value	Detail
config-location	ex) classpath:mybatis-config.xml	mybatis 설정파일 경로를 지정합니다.

mybatis-config.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true" />
        <setting name="useGeneratedKeys" value="true" />
        <setting name="defaultExecutorType" value="BATCH" />
        <setting name="defaultStatementTimeout" value="10000" />
    </settings>
    <mappers>
      <mapper resource="SQL_IF_REST_DB_01.xml" />
      <mapper resource="SQL_IF_REST_DB_02.xml" />
      <mapper resource="SQL_IF_REST_DB_03.xml" />
      <mapper resource="SQL_IF_REST_DB_04.xml" />
    </mappers>
</configuration>
```

Mybatis를 사용하기위한 설정파일입니다. 인터페이스가 추가될때마다 mapper를 추가합니다.

SQL_IF_REST_DB_01.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_01">
   <insert id="INSERT" parameterType="hashmap">
        INSERT INTO COMPANY_DATA(ID, NAME, COMPANY_NAME, APP_NAME,
DOMAIN_NAME)
        values (
                 SEQ_COMPANY_DATA.NEXTVAL ,
                 #{name     ,jdbcType=VARCHAR},
                 #{companyName     ,jdbcType=VARCHAR},
                 #{appName     ,jdbcType=VARCHAR},
                 #{domainName     ,jdbcType=VARCHAR}
                 )
    </insert>
</mapper>
```

namespace는 인터페이스 ID를 넣고 INSERT 를 추가합니다.

SQL_IF_REST_DB_02.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_02">
   <select id="SELECT" parameterType="hashmap" resultType="hashmap">
        SELECT * FROM COMPANY_DATA
        WHERE NAME LIKE '%'||#{name}||'%'
    </select>
</mapper>
```

namespace는 인터페이스 ID를 넣고 SELECT 를 추가합니다.

SQL_IF_REST_DB_03.xml

Mybatis 설정 파일

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_03">
    <update id="UPDATE" parameterType="hashmap">
        UPDATE COMPANY_DATA
        SET
        <trim suffixOverrides=",">
           <if test="companyName != null and companyName != ''"
>COMPANY_NAME =  #{companyName,jdbcType=VARCHAR} ,</if>
           <if test="appName != null and appName != ''" >APP_NAME =
#{appName,jdbcType=VARCHAR}, </if>
           <if test="domainName != null and domainName != ''" >DOMAIN_NAME
= #{domainName,jdbcType=VARCHAR}, </if>
           <if test="name != null and name != ''" >NAME =
#{name,jdbcType=VARCHAR}, </if>
        </trim>
        WHERE ID = #{id}
    </update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 UPDATE 를 추가합니다.

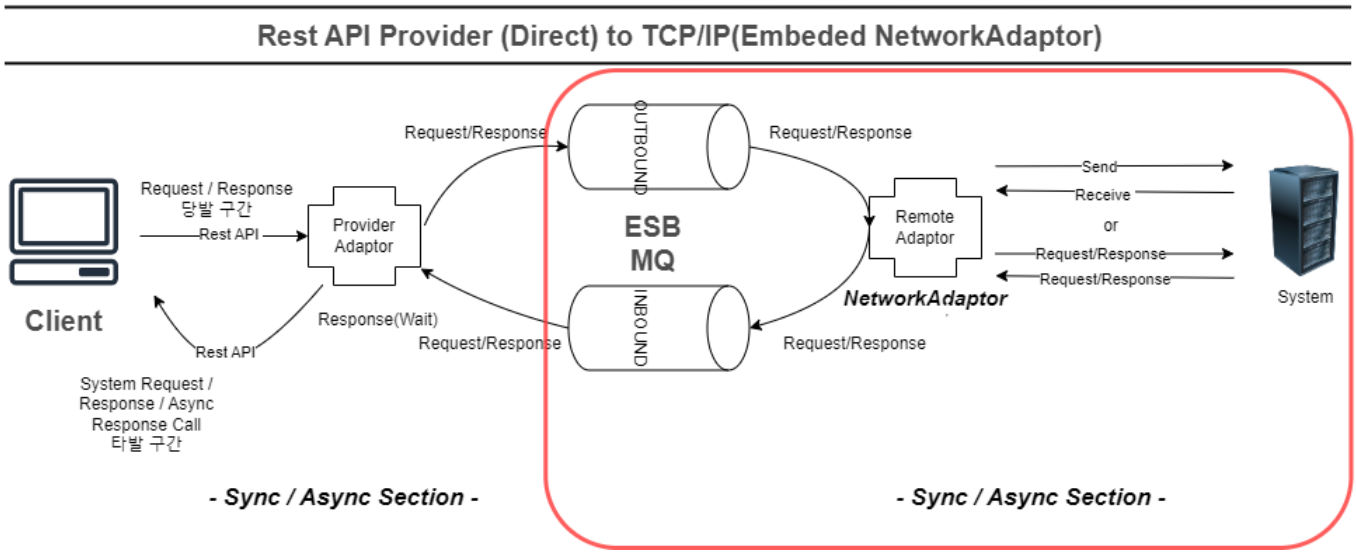
SQL_IF_REST_DB_04.xml

Mybatis 설정 파일

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="IF_REST_DB_04">
    <update id="DELETE" parameterType="hashmap">
        DELETE COMPANY_DATA
        WHERE ID = #{id}
    </update>
</mapper>
```

namespace는 인터페이스 ID를 넣고 DELETE 를 추가합니다.

12.4 REMOTE TCP/IP 처리 구성도(우측 Remote Adaptor)



★ Remote 어댑터는 ESB MQ (Outbound) 로 부터
1. 요청 메시지를 받아 대상(Target) 시스템에 전문을 전달(Send Or Request)합니다.
2. 대상(Target) 시스템으로부터 전문을 수신(Receive)받으면 ESB MQ(Inbound)로 전달합니다.

REST API 프로바이더 DIRECT TCP/IP 방식에서 Provider와 Network처리가 분리되어 위의 그림처럼 우측 영역만 수행을 합니다.
진행 순서가 없는건 단방향이 아닌 Client에서 시작이되는 당발과 System에서 시작이 되는 타발로 양방향 통신을 구성하기 때문입니다.

12.5 NA_REMOTE_TCP_01 : REMOTE TCP/IP

- 필수 공통 리소스

JmsConnectionFactory

JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

application.yml

어플리케이션 설정 파일

```yaml
network:
  adaptor:
    tcp-message:
      length:
        enabled: true
        alias: SGI_TCP
        jms-listener-use: true
        jms-listener-queue-name: OUT.SGI_TCP
        jms-concurrent-consumers: 10
        call-type: JMS
        callback-queue-name: IN.SGI_TCP
        callback-receive-timeout: 5000
        tcp-mode: SYNC
        remote-hosts:
          - 127.0.0.1
        remote-ports:
          - 8004
        extractor:
          message-id: 39
          message-id-len: 20
          work-code: 24
          work-code-len: 7
        hex-mode: false
        length-field-offset: 0
        length-field-length: 4
        length-adjustment: 0
        initial-bytes-to-strip: 0
        string-value-use: true
        max-frame-length: 4096
        reconnect-sec: 30
        recv-task-count: 10
        request-usage-size: 10000
        request-wait-time-out-sec: 10
        src-org: A01
        trg-org: SCI
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
  autoconfigure:
    exclude:
      -
org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfigurati
on
      -
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

Table 71. network.adaptor.tcp-message.length

Key	Value	Detail
enabled	true, false	네트워크 어뎁터 길이필드 사용여부
alias	ex) SGI_TCP	별칭
jms-listener-use	true, false	JMS 리스너 사용여부
jms-listener-queue-name	OUT.SGI_TCP	JMS 리스너 큐명
jms-concurrent-consumers	ex) 10	JMS 리스너 동시처리자 갯수
call-type	ex) JMS	호출 타입 (JMS, HTTP)
callback-queue-name	IN.SGI_TCP	콜백 큐명
callback-receive-timeout	ex) 5000	콜백 수신 타임아웃
tcp-mode	ex) SYNC	TCP/IP 통신 모드 (SYNC, ASYNC)
remote-hosts	ex) 127.0.0.1	대상 시스템 IP
remote-ports	ex) 8004	대상 시스템 PORT
hex-mode	true, false	16진수 모드 여부 (default : false)
length-field-offset	ex) 0	길이 필드 오프셋
length-field-length	ex) 4	길이 필드 길이
length-adjestment	ex) 0	길이 조정
initial-bytes-to-strip	ex) 0	초기 바이트 제거
string-value-use	true, false	문자열 사용 여부
max-frame-length	ex) 4096	최대 프레임 길이
reconnect-sec	ex) 30	재연결 시간
recv-task-count	ex) 10	수신 작업자 갯수
request-usage-size	ex) 10000	요청 사용 크기
request-wait-time-out-sec	ex) 10	요청 대기 타임아웃 시간
src-org	ex) A01	송신기관(로그 이력용)
trg-org	ex) SCI	수신기관(로그 이력용)

Table 72. network.adaptor.tcp-message.length.extractor

Key	Value	Detail
message-id	ex) 39	메시지 ID(요청과 응답의 메시지가 짝이 되어야할 고유한값의 index) ex)전문거래번호
message-id-len	ex) 20	메시지 ID 길이
work-code	ex) 24	업무를 식별할수있는 업무구분코드 인덱스
work-code-len	ex) 7	업무구분코드 길이

Table 73. spring.main

Key	Value	Detail
web-application-type	none, reactive, servlet	web-application 을 포함하지 않을경우 none으로 입력합니다.
allow-bean-definition-overriding	true, false	구동시에 bean이 같은 id가 존재할경우 오버라이딩 할건지에 대한 내용이며 이중화 관련 재기동 동작기능이 포함되어있어서 해당옵션을 true로 고정합니다.

Table 74. spring.autoconfigure

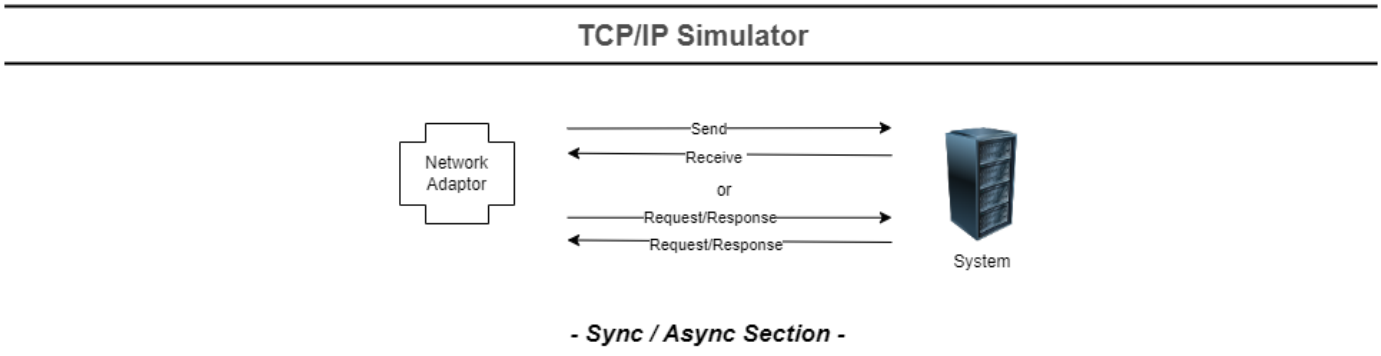
Key	Value	Detail
exclude	ex) org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration	
exclude	ex) org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration	제외할 설정

13 시뮬레이터

13.1 개요

시뮬레이터

13.2 TCP/IP 처리 구성도



	★ 어뎁터는 대상 System의 시뮬레이터가 되어 1. 요청 메시지를 받아 가상전문을 응답합니다. 2. 통신구성은 NetworkAdaptor와 동일한 설정을 가지고 있습니다.

13.3 NA_TCP_SIMUL_01 : TCP/IP SIMULATOR

- 필수 공통 리소스

  JmsConnectionFactory

  JMS 설정 (jmsTemplate 포함) , id를 jmsConnectionFactory로 등록하면 자동 Injection

- 필수 설정 파일

  application.yml

  어플리케이션 설정 파일

```yaml
network:
  simulator:
    tcp-message:
      length:
        enabled: true
        tcp-mode: sync
        bind-ports:
          - 8004
        extractor:
           message-id: 39
           message-id-len: 20
           work-code: 24
           work-code-len: 7
        connect-timeout-sec: 30
        hex-mode: false
        initial-bytes-to-strip: 0
        length-adjustment: 0
        length-field-length: 4
        length-field-offset: 0
        max-frame-length: 4096
        reconnect-sec: 30
        recv-task-count: 2
        string-value-use: true
spring:
  autoconfigure:
    exclude:
      -
org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfigurati
on
      -
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
server:
  port: 24057
"0200295": "[0340HEHY001
SHDI0020000002102950000001C000000000000000000001234567890123456789012345678
901234567890123456789012345678901234112345678901234567890123456789012345678
901234567890123456789012345678901234567890123456789012345678901234567890123
456789012345678901234567890123456789012345678901234567890123456789012345678
901234567890123456789012345678901234]"
```

Table 75. network.simulator.tcp-message.length

Key	Value	Detail
enabled	true, false	네트워크 시뮬레이션 어뎁터 길이필드 사용여부
tcp-mode	ex) sync	TCP/IP 통신 모드 (SYNC, ASYNC)

Key	Value	Detail
bind-ports	ex) 8004	바인드 포트
connect-timeout-sec	ex) 30	연결 타임아웃 시간
hex-mode	ex) true, false	16진수 모드 여부 (default : false)
initial-bytes-to-strip	ex) 0	초기 바이트 제거
length-adjustment	ex) 0	길이 조정
length-field-length	ex) 4	길이 필드 길이
length-field-offset	ex) 0	길이 필드 오프셋
max-frame-length	ex) 4096	최대 프레임 길이
reconnect-sec	ex) 30	재연결 시간
string-value-use	true, false	문자열 사용 여부

Table 76. network.simulator.tcp-message.length.extractor

Key	Value	Detail
message-id	ex) 39	메시지 ID(요청과 응답의 메시지가 짝이 되어야할 고유한값의 index) ex)전문거래번호
message-id-len	ex) 20	메시지 ID 길이
work-code	ex) 24	업무를 식별할수있는 업무구분코드 인덱스
work-code-len	ex) 7	업무구분코드 길이

Table 77. spring.autoconfigure

Key	Value	Detail
exclude	ex) org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration	제외할 설정
exclude	ex) org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration	제외할 설정

Table 78. server

Key	Value	Detail
port	ex) 24057	HTTP(HTTPS) 서비스 포트
"0200295"	ex) "[0340HEHY001 …]"	가상응답 메시지 (work-code : [전문내용])

14 인터페이스 등록 및 Adaptor 연동

Application.yml에 인터페이스 설정을 인터페이스 등록/조회를 통하여 등록을 할수 있게 되었습니다. *v1.0.2부터 지원*

14.1 auto-config 활성화

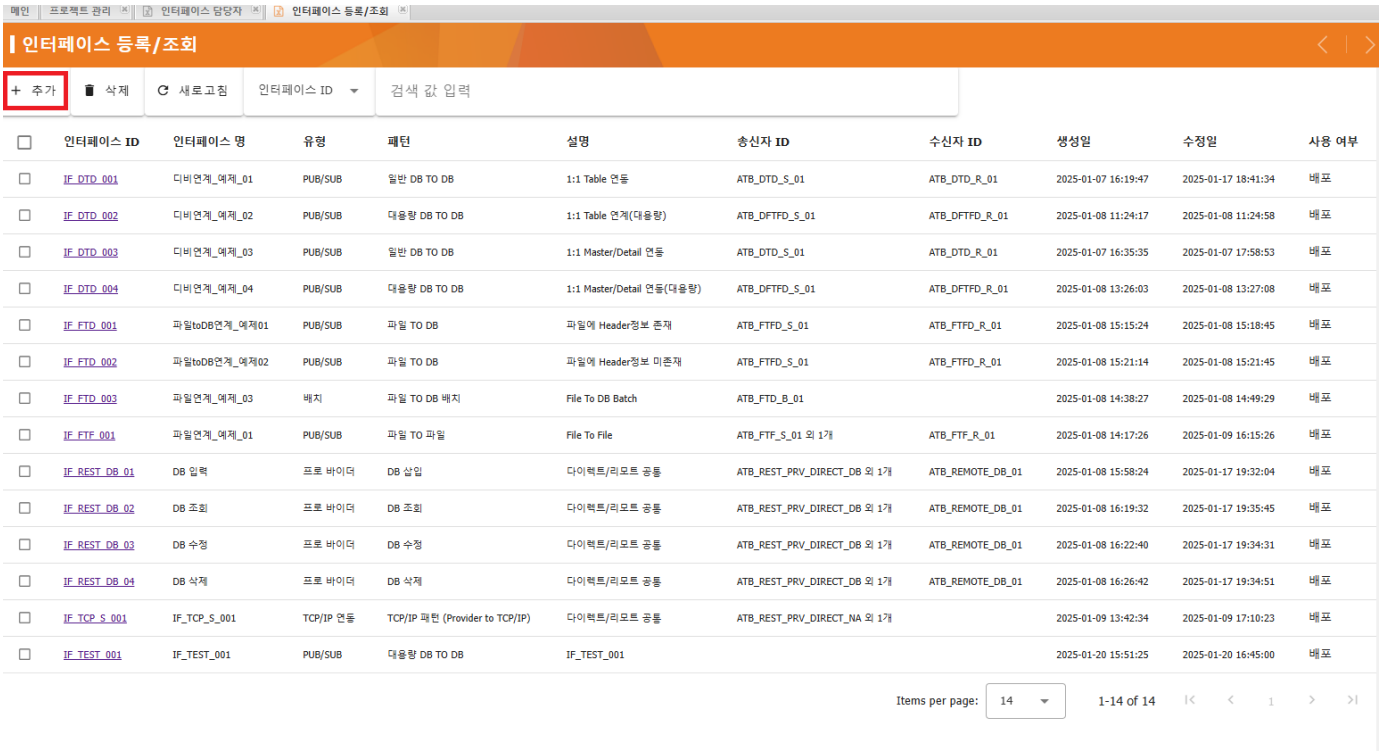
context 설정을 제거하고 auto-config를 true로 설정하면 Adaptor 구동시에 인터페이스 설정을 가져옵니다.
아래는 설정 예제입니다.

```yaml
atb:
  if-config:
    id: DTD_SENDER_01
    description: "(DB SENDER) + JMS + (DB RECEIVER)"
    prefix: DTD
    # 인터페이스 관리 설정에서 가져옵니다.
    auto-config: true
spring:
  main:
    web-application-type: none
    allow-bean-definition-overriding: true
mybatis:
  config-location: classpath:mybatis-config.xml
```

14.2 인터페이스 등록

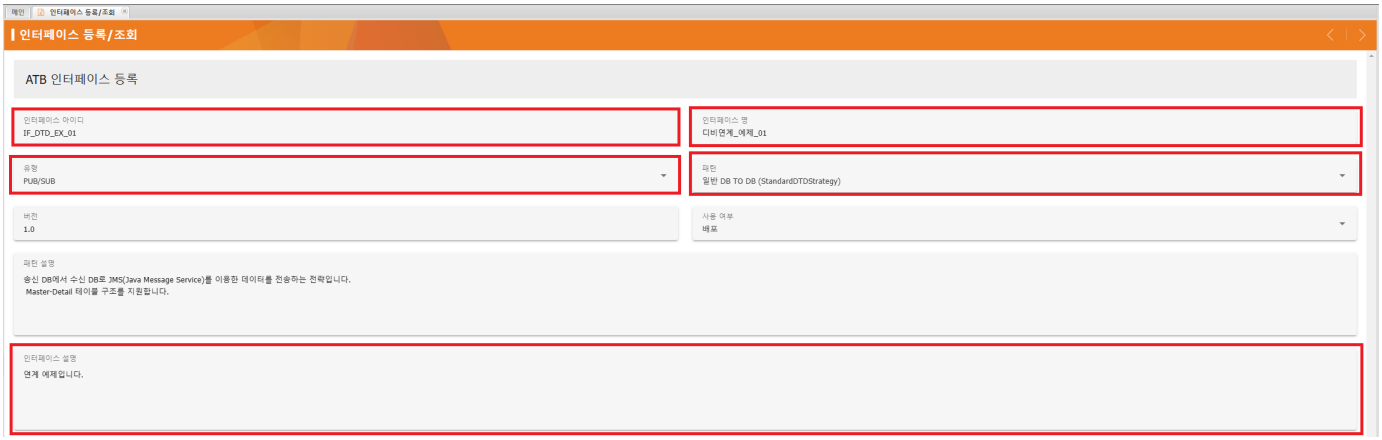
인터페이스 등록은 인터페이스 등록/조회 화면으로 이동후 아래와 같이 등록을 합니다.

14.2.1 추가 버튼 선택

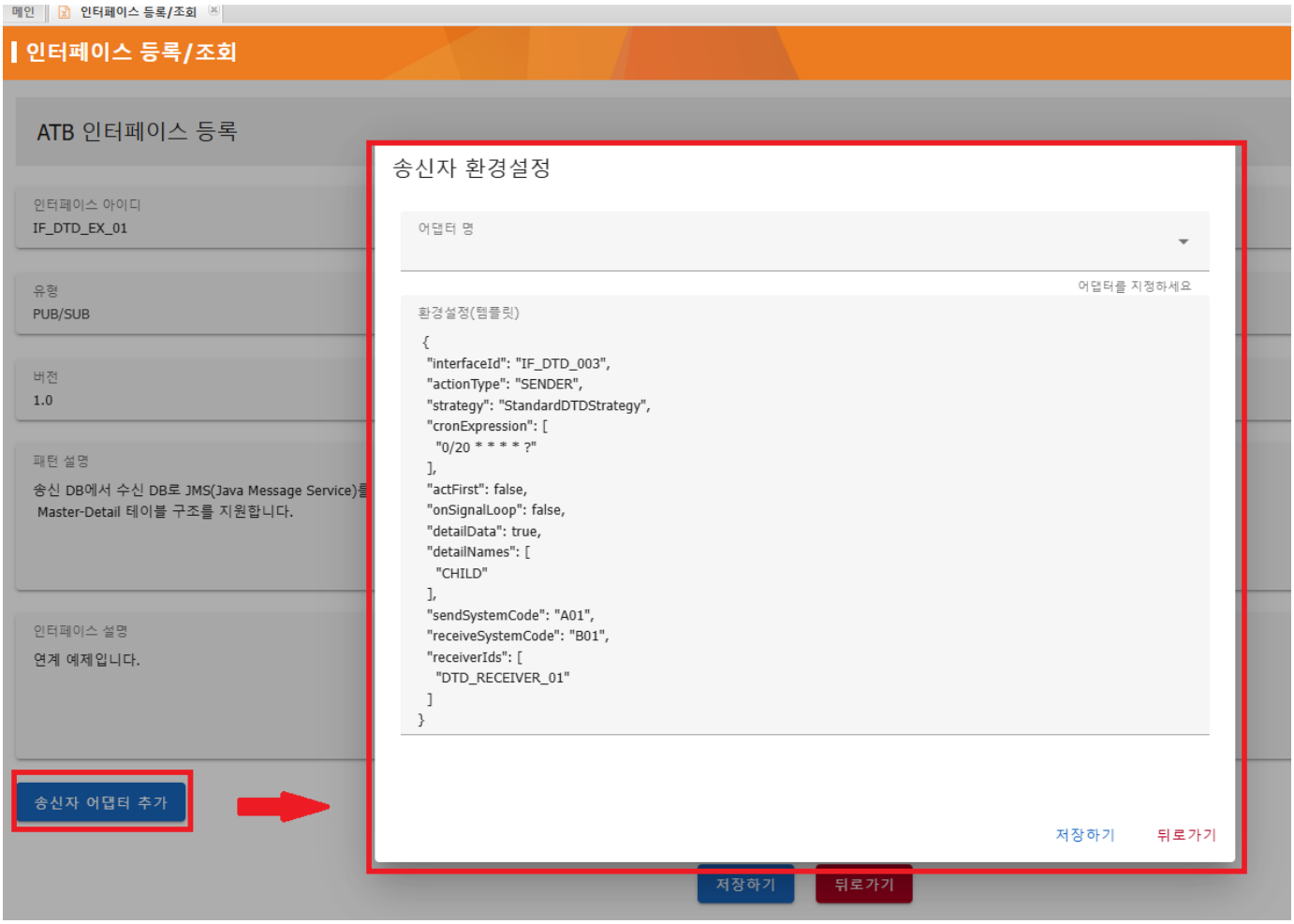


인터페이스 ID	인터페이스 명	유형	패턴	설명	송신자 ID	수신자 ID	생성일	수정일	사용 여부
IF_DTD_001	디비연계_예제_01	PUB/SUB	일반 DB TO DB	1:1 Table 연동	ATB_DTD_S_01	ATB_DTD_R_01	2025-01-07 16:19:47	2025-01-17 18:41:34	배포
IF_DTD_002	디비연계_예제_02	PUB/SUB	대용량 DB TO DB	1:1 Table 연계(대용량)	ATB_DFTFD_S_01	ATB_DFTFD_R_01	2025-01-08 11:24:17	2025-01-08 11:24:58	배포
IF_DTD_003	디비연계_예제_03	PUB/SUB	일반 DB TO DB	1:1 Master/Detail 연동	ATB_DTD_S_01	ATB_DTD_R_01	2025-01-07 16:35:35	2025-01-07 17:58:53	배포
IF_DTD_004	디비연계_예제_04	PUB/SUB	대용량 DB TO DB	1:1 Master/Detail 연동(대용량)	ATB_DFTFD_S_01	ATB_DFTFD_R_01	2025-01-08 13:26:03	2025-01-08 13:27:08	배포
IF_FTD_001	파일toDB연계_예제01	PUB/SUB	파일 TO DB	파일에 Header정보 존재	ATB_FTFD_S_01	ATB_FTFD_R_01	2025-01-08 15:15:24	2025-01-08 15:18:45	배포
IF_FTD_002	파일toDB연계_예제02	PUB/SUB	파일 TO DB	파일에 Header정보 미존재	ATB_FTFD_S_01	ATB_FTFD_R_01	2025-01-08 15:21:14	2025-01-08 15:21:45	배포
IF_FTD_003	파일연계_예제_03	배치	파일 TO DB 배치	File To DB Batch	ATB_FTD_B_01		2025-01-08 14:38:27	2025-01-08 14:49:29	배포
IF_FTF_001	파일연계_예제_01	PUB/SUB	파일 TO 파일	File To File	ATB_FTF_S_01 외 1개	ATB_FTF_R_01	2025-01-08 14:17:26	2025-01-09 16:15:26	배포
IF_REST_DB_01	DB 입력	프로 바이더	DB 삽입	다이렉트/리모트 공통	ATB_REST_PRV_DIRECT_DB 외 1개	ATB_REMOTE_DB_01	2025-01-08 15:58:24	2025-01-17 19:32:04	배포
IF_REST_DB_02	DB 조회	프로 바이더	DB 조회	다이렉트/리모트 공통	ATB_REST_PRV_DIRECT_DB 외 1개	ATB_REMOTE_DB_01	2025-01-08 16:19:32	2025-01-17 19:35:45	배포
IF_REST_DB_03	DB 수정	프로 바이더	DB 수정	다이렉트/리모트 공통	ATB_REST_PRV_DIRECT_DB 외 1개	ATB_REMOTE_DB_01	2025-01-08 16:22:40	2025-01-17 19:34:31	배포
IF_REST_DB_04	DB 삭제	프로 바이더	DB 삭제	다이렉트/리모트 공통	ATB_REST_PRV_DIRECT_DB 외 1개	ATB_REMOTE_DB_01	2025-01-08 16:26:42	2025-01-17 19:34:51	배포
IF_TCP_S_001	IF_TCP_S_001	TCP/IP 연동	TCP/IP 패턴 (Provider to TCP/IP)	다이렉트/리모트 공통	ATB_REST_PRV_DIRECT_NA 외 1개		2025-01-09 13:42:34	2025-01-09 17:10:23	배포
IF_TEST_001	IF_TEST_001	PUB/SUB	대용량 DB TO DB	IF_TEST_001			2025-01-20 15:51:25	2025-01-20 16:45:00	배포

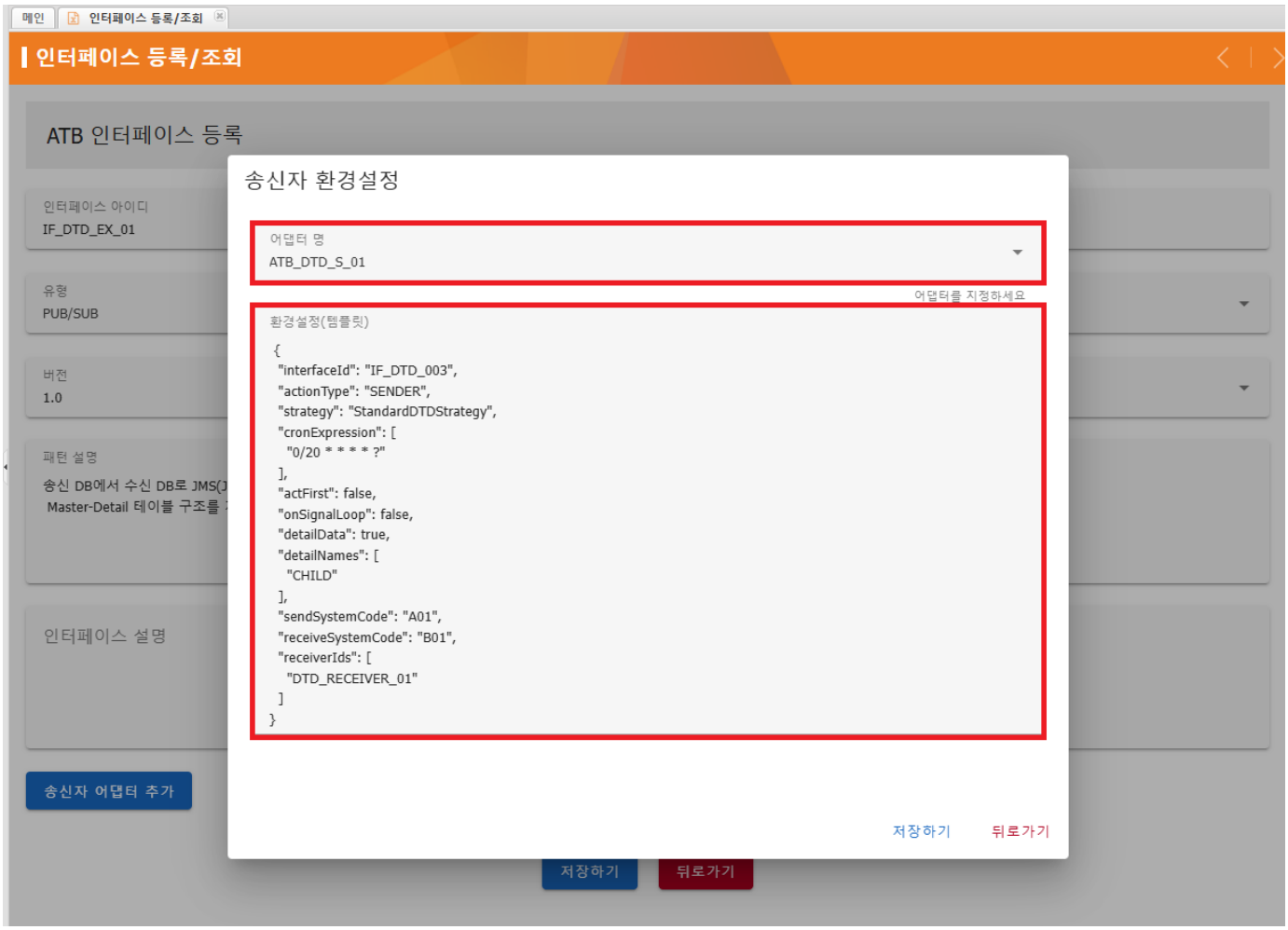
14.2.2 인터페이스 정보 입력



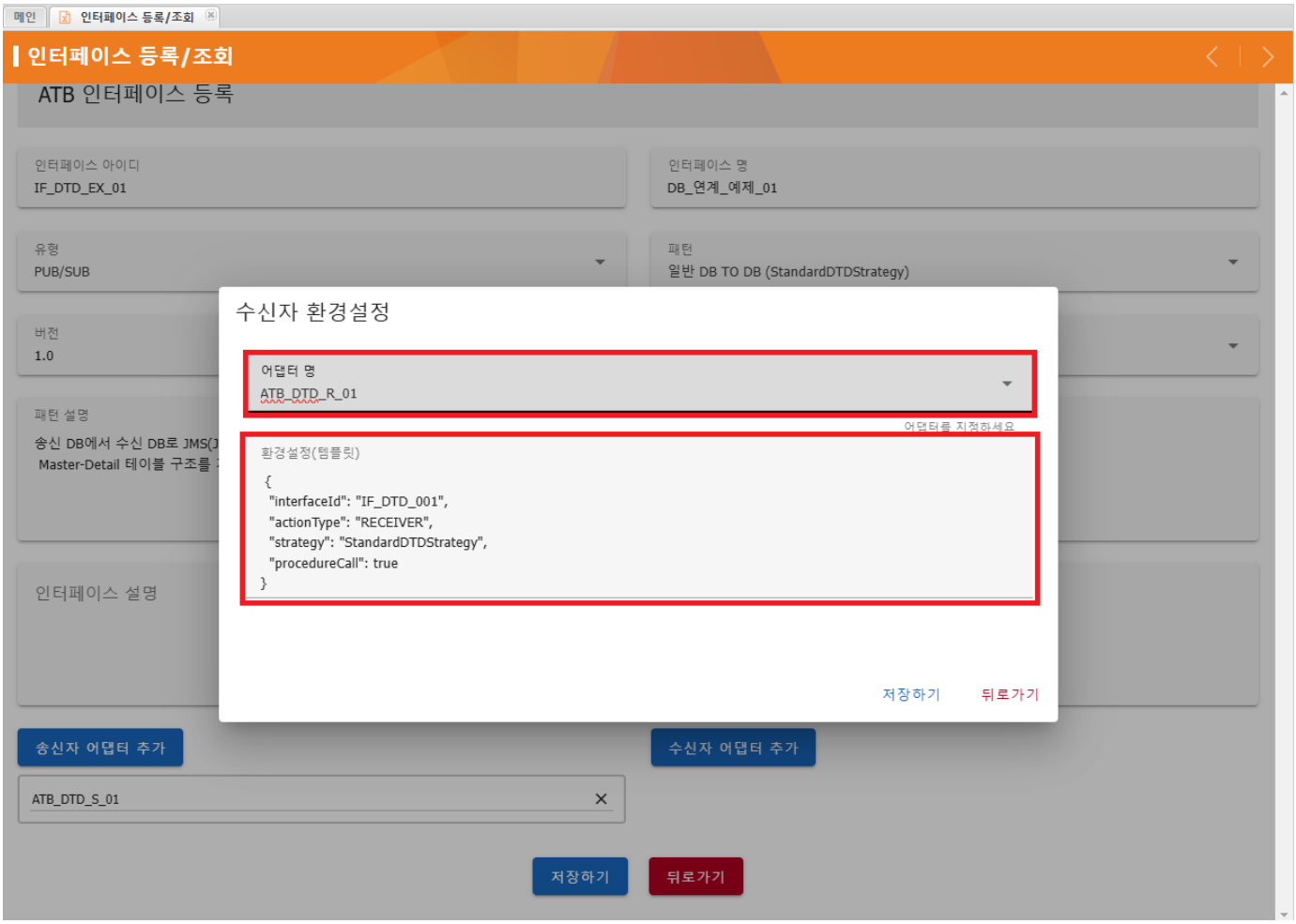
14.2.3 송신자 어댑터 추가 → 송신자 환경설정



14.2.4 어뎁터 지정 → 환경설정(템플릿) 수정



14.2.5 수신자 어댑터 추가 → 수신자 환경설정


메인
인터페이스 등록/조회
인터페이스 등록/조회
ATB 인터페이스 등록
인터페이스 아이디
IF_DTD_EX_01
인터페이스 명
DB_연계_예제_01
유형
PUB/SUB
패턴
일반 DB TO DB (StandardDTDStrategy)
버전
1.0
패턴 설명
송신 DB에서 수신 DB로 JMS(J
Master-Detail 테이블 구조를
인터페이스 설명
송신자 어댑터 추가
수신자 어댑터 추가
ATB_DTD_S_01
저장하기
뒤로가기
수신자 환경설정
어댑터 명
ATB_DTD_R_01
어댑터를 지정하세요
환경설정(템플릿)
{
"interfaceId": "IF_DTD_001",
"actionType": "RECEIVER",
"strategy": "StandardDTDStrategy",
"procedureCall": true
}
저장하기
뒤로가기